

Bienvenue au cours

PYTHON ADVANCED

Programmation avancée avec Python



Formateur: Philippe Ney

Digital Competence. Made of People.





Python Advanced

Programmation avancée avec Python

Philippe Ney
philippe.ney@lilabyte.com



Présentation

► Formation

- Master en Microtechnique, EPFL, Lausanne

► Expérience professionnelle

- 1996-1999 Assistant EPFL, Laboratoire de Vision Industrielle
- 1999-2000 Chef de projet, Namics SA, Lausanne
- 2000-2003 Développeur software, Smartdata SA, Lausanne
- 2002-2004 Fondateur, TiZoo Sàrl, Hauterive
- 2004-2005 Développeur software, Inforlearn, Lausanne
- 2005-2007 CTO, Pardes Group SA, Fribourg
- 2007-2020 TiZoo Sàrl, Hauterive
- Dès 2020 LilaByte Sàrl, Lausanne

► LilaByte Sàrl

- Administration système & intégration linux.
- Développement d'applications & Web.
- Hébergement data cloud & mails.
- Enseignement linux, unix & développement en collaboration avec Digicomp.





Plan du cours

► Premier jour

- Formats de communication
- *Packing et unpacking*
- *Duck typing*
- Programmation fonctionnelle
- Manipulations de dictionnaires
- Typer le code Python
- Modules et packages

► Deuxième jour

- Programmation objet
 - Les bases
- Gestion des exceptions
- Clause **else** des boucles
- Décorateurs
- Programmation objet
 - Notions avancées
- Profiling et debugging
- Testing



Supports

- ▶ Le cours au format PDF, les données utilisées dans les exemples et dans les exercices ainsi que les corrigés des exercices se trouvent ici :
- ▶ <https://doc.lilabyte.com/pyadv>
- ▶ Username/password : `pyadv` / `pyadv`



Format de communication

JSON

XML



Introduction

- ▶ Aujourd'hui les 2 principaux format d'échange de données dans le cas des web services sont **XML** (*eXtensible Markup Language*) et **JSON** (*JavaScript Object Notation*).
- ▶ XML est ancien, bien établi et largement utilisé.
 - ▶ Mais les APIs qui utilisent XML, tel que SOAP, sont vieilles et quelque peu complexes à implémenter.
- ▶ JSON étant plus simple, plus concis, plus rapide à analyser et correspondant souvent à des structures existantes dans beaucoup de langages Web, il tend à remplacer XML dans les applications Web.
 - ▶ En particulier lors de communication avec JavaScript.
 - ▶ Typiquement dans le cas d'une API de type REST.
- ▶ JSON correspond à un dictionnaire Python, ce qui fait que la conversion entre les deux formats est très simple.



JSON

- ▶ En JSON les tableaux utilisent les crochets (`[]`), comme les listes Python et les objets ou les tableaux associatifs utilisent les accolades (`{}`) comme les dictionnaires Python.
- ▶ La différence est qu'en JSON les clés et les valeurs de type **string** sont **toujours** entourées de guillemets doubles.
- ▶ Le module **json** fournit les méthodes suivantes :

<code>dump(obj, fp)</code> <code>dumps(obj)</code>	Encode un objet Python en JSON dans un fichier. Ou dans une string qui est retournée.
<code>load(fp)</code> <code>loads(s)</code>	Décode un objet JSON depuis un fichier. Ou depuis une string.



XML

- ▶ La manière la plus simple de traiter le XML avec Python est d'utiliser le module `xml.etree.ElementTree` (**ET**).
- ▶ Considérons le fichier XML de la slide suivante.
- ▶ Le fichier XML est chargé à l'aide de la méthode `parse()` puis la méthode `getroot()` permet de récupérer l'élément racine :

```
import xml.etree.ElementTree as ET
tree = ET.parse("cave.xml")
root = tree.getroot()
```

- ▶ `tree` est un objet de type `ElementTree` qui représente l'entier du document sous forme d'un arbre.
- ▶ `root` est un objet de type `Element` qui représente un nœud de l'arbre.
- ▶ Il est aussi possible de charger du XML depuis une `string` :

```
root = ET.fromstring(string_with_xml_data)
```

- ▶ Dans ce cas on récupère directement le nœud racine.



XML

```
<?xml version="1.0" encoding="UTF-8"?>
<cave>
  <vin type="rouge" stock="2">
    <pays>France</pays>
    <region>Bordeau</region>
    <type>Saint-Emilion</type>
    <annee>1998</annee>
    <nom>Château Figeac</nom>
  </vin>
  <vin type="blanc" stock="1">
    <pays>Suisse</pays>
    <region>Vaud</region>
    <type>La Côte</type>
    <annee>2011</annee>
    <nom>Domaine de Vinzel</nom>
  </vin>
  <biere type="blonde" stock="7">
    <pays>Suisse</pays>
    <region>Vaud</region>
    <contenance>33cl</contenance>
    <nom>Boxer Old</nom>
  </biere>
</cave>
```



XML

- ▶ Chaque nœud, de type **Element**, possède les attributs **tag** et **attrib** :

```
root.tag  
'cave'  
root.attrib  
{}
```

- ▶ Le parcours des éléments fils d'un nœud se fait très simplement :

```
for bottle in root:  
    print(bottle.tag, bottle.attrib)  
vin {'type': 'rouge', 'stock': '2'}  
vin {'type': 'blanc', 'stock': '1'}  
biere {'type': 'blonde', 'stock': '7'}
```

- ▶ Les nœuds fils sont imbriqués et sont donc accessibles par leur index :

```
root[2][3].text  
'Boxer Old'
```



XML

- Les méthodes `find()` et `findall()` permettent de trouver le premier ou tous les éléments fils en fonction de leur `tag` :

```
for bottle in root.findall('vin'):  
    pays = bottle.find('pays').text  
    name = bottle.find('nom').text  
    print(pays, name)
```

```
France Château Figeac  
Suisse Domaine de Vinzel
```

- L'attribut `text` permettra d'accéder au contenu textuel de l'élément.



XML

- ▶ L'accès aux attributs se fait à l'aide de la méthode `get()`, ou en utilisant l'accès par dictionnaire.
- ▶ `get()` permet de définir une valeur par défaut qui sera retournée si l'élément ne contient pas l'attribut :

```
v = root.find('vin')
v.attrib
{'type': 'rouge', 'stock': '2'}
v.attrib['stock']
'2'
v.get('stock', 22)
'2'
v.get('stroc', 0)
0
```



XPath

- ▶ Il est également possible d'accéder aux informations du fichier XML en utilisant des expressions XPath à l'aide des méthodes `find()` ou `findall()`.
- ▶ Pour récupérer les noms de toutes les boissons de la cave :

```
for name in root.findall('*/*nom'):  
    print(name.text)  
Château Figeac  
Domaine de Vinzel  
Boxer Old
```

- ▶ Le support de XPath est cependant limité.



Modifications et sauvegarde

- ▶ **ET** fournit les méthodes permettant de modifier le XML :
 - ▶ Un carton de Vinzel 2018 vient d'être livré.
 - ▶ Il n'y a plus de Château Figeac et il n'en sera pas recommandé.

```
for wine in root.findall('./vin'):  
    if wine.find('./nom').text == "Domaine de Vinzel":  
        annee = wine.find('./annee')  
        # Modify the text of a node  
        annee.text = '2018'  
        stock = int(wine.attrib['stock'])  
        stock += 12  
        # Modify the value of an attribute  
        wine.set('stock', str(stock))  
  
    elif wine.find('./nom').text == "Château Figeac":  
        root.remove(wine)
```



Modifications et sauvegarde

- Livraison d'une caisse de Château Cheval Blanc 2017 :

```
cheval = ET.SubElement(root, 'vin',  
                        {'type': "rouge", 'stock': "6"})  
ET.SubElement(cheval, 'pays').text = "France"  
ET.SubElement(cheval, 'region').text = "Bordeau"  
ET.SubElement(cheval, 'type').text = "Saint-Emilion"  
ET.SubElement(cheval, 'annee').text = "2017"  
ET.SubElement(cheval, 'nom').text = "Château Cheval Blanc"
```

- Et finalement, sauvegarde du fichier décrivant le nouveau contenu de la cave en s'assurant de conserver l'encodage avec les accents :

```
tree.write("cave.xml", encoding="utf-8")
```



Packing et unpacking (emballer et déballer)



Introduction

- ▶ **Unpacking** Transformer un itérable en autant d'éléments disjoints qu'il contient (déballer).
- ▶ **Packing** Transformer des éléments disjoints en un itérable (emballer).
- ▶ Cette logique s'applique également aux dictionnaires.
- ▶ Ce qui permet de passer une liste à une fonction qui attend plusieurs paramètres :
 - ▶ Soit la fonction `do_smth` définie de la manière suivante :

```
def do_smth(a, b, c):  
    print(a, b, c)
```

- ▶ L'appel suivant n'est pas possible et génère un erreur:

```
data = [1, 2, 3]  
do_smth(data)
```

- **`TypeError: do_smth() missing 2 required positional arguments: 'b' and 'c'`**



Unpacking

- ▶ Ce qui serait cool est que l'appel de la slide précédente puisse fonctionner automatiquement et associer chaque élément de la liste à un paramètre de la fonction de manière positionnelle.
- ▶ C'est ce que permet justement l'opérateur `*` qui permet de débiller la liste.
- ▶ L'appel suivant fonctionnera parfaitement :

```
data = [1, 2, 3]  
do_smth(*data)
```

- ▶ Bien entendu il faut que la longueur de la liste corresponde au nombre d'arguments, sinon une erreur sera générée.



Unpacking

- Le déballage se fait souvent de manière automatique :

```
a, b = range(2)
=> a = 0 et b = 1

hello_list = ["Hello", "World"]
h, w = hello_list
=> h = 'Hello' et w = 'World'

def get_data():
    person = ("James", "Bond")      # Returned by the db
    return person

first, name = get_data()
print(f"My name is {name}, {first} {name}")
My name is Bond, James Bond
```



Unpacking d'un dictionnaire

- ▶ Le déballage fonctionne-t-il aussi si les données fournies à la fonction `do_smth` proviennent d'un dictionnaire ?

```
data = {'a': 1, 'b': 2, 'c': 3}
do_smth(*data)
a b c
```

- ▶ Oups ! On dirait bien que ça ne fonctionne pas.
 - ▶ En fait si, ça fonctionne très bien, mais pour un dictionnaire, l'astérisque simple (*) permet de déballer **les clés**.
 - Ce qui peut également être intéressant.
- ▶ Pour déballer les valeurs il faut doubler l'astérisque (**):

```
data = {'a': 1, 'b': 2, 'c': 3}
do_smth(**data)
1 2 3
```



Unpacking

- ▶ Le déballage est très intéressant pour :
 - ▶ Concaténer des listes de manière aléatoire.
 - ▶ Concaténer ou mettre à jour des dictionnaires.

```
l1 = [1, 2]
l2 = ["a", "b"]
long_list = [*l1, *l2, *l1]
[1, 2, 'a', 'b', 1, 2]

julien = {'first': "Julien", 'name': "Ambord"}
info = {'email': "julien@poy.ch"}
julien = {**julien, **info}
{'first': 'Julien', 'name': 'Ambord', 'email': 'julien@poy.ch'}

update = {'email': "julien@yop.com"}
julien = {**julien, **update}
{'first': 'Julien', 'name': 'Ambord', 'email': 'julien@yop.com'}
```



Packing

- ▶ Selon la même logique il est également possible d'emballer automatiquement des éléments disjoints :

```
james = first, name  
james  
( 'James' , 'Bond' )
```

- ▶ Ce qui produit un **tuple**.



Packing

- ▶ L'opérateur `*` peut également être utilisé pour emballer des éléments, et permet d'effectuer des manipulations intéressantes :

```
a, b, *rest = range(5)
rest
[2, 3, 4]
```

- ▶ La variable (ici `rest` de type `list`) peut se placer n'importe où :

```
a, *rest, b = range(5)
rest
[1, 2, 3]
*rest, a, b = range(5)
rest
[0, 1, 2]
```

- ▶ Par exemple, récupérer la première et la dernière ligne d'un fichier :

```
with open("file.txt") as fd:
    first, *_ , last = fd.readlines()
```



Refactorisation des arguments

- ▶ Avec l'emballage il est très facile de définir un nombre variable d'arguments pour une fonction :

```
def do_something(*args):  
    a, b, *others = args  
    # Some clever processing
```

- ▶ Si l'on veut forcer l'utilisateur à fournir au moins deux paramètres il suffit de les définir de manière explicite :

```
def do_something(a, b, *args):  
    # Some clever processing
```

Paramètres accessibles uniquement par leur nom



- Pour définir une fonction avec des paramètres nommés mais accessibles uniquement par leur nom on utilise :

```
def do_something(a, b, *args, option=True):  
    ...
```

- Comme le paramètre `option` vient après `*args`, la seule manière de lui assigner un argument est d'utiliser son nom :
 - Sinon tous les arguments positionnels seront automatiquement intégrés dans la variable `args` (de type `tuple`).

```
do_something(x, y, option=False)
```

- Dans un tel cas, les paramètres optionnels doivent évidemment tous posséder une valeur par défaut.

Paramètres accessibles uniquement par leur nom



- ▶ Si les valeurs récupérées par la variable **args** ne sont pas utiles il suffit de mettre uniquement l'astérisque ***** :

```
def do_smth(a, b, *, option=True):  
    ...
```

- ▶ De cette manière la fonction prend réellement uniquement 2 arguments positionnels et un argument nommé.
 - ▶ Un appel avec 3 arguments positionnels générera une erreur.
- ▶ Finalement pour créer une fonction qui n'accepte que des arguments nommés :

```
def do_smth(*, start=0, stop=10):  
    ...
```



Packing de dictionnaires

- ▶ Pour emballer les arguments d'une fonction dans un dictionnaire la logique est identique, mais en utilisant le double astérisque (******) qui cible les dictionnaires.
- ▶ Les arguments devront être nommés, sinon l'association clé ↔ valeur sera impossible.

```
def do_something(**kwargs):  
    for key in kwargs:  
        print(f"{key} => {kwargs[key]}")  
  
do_something(first="Julien", name="Ambord")  
  
first => Julien  
name => Ambord
```



Exercices 1

- ▶ Testez la récupération des arguments positionnels d'une fonction avec ****kwargs** et testez la récupération des arguments nommés avec ***args**.
- ▶ Des résultats que vous avez obtenus, définissez une fonction pouvant récupérer des arguments positionnels et des arguments nommés.



Duck typing



Introduction

- ▶ Le *duck typing* est une manière de typer les données où le type d'un objet est moins important que les méthodes que l'objet définit.
- ▶ Suivant le *duck typing* le type d'un objet n'est pas testé, mais c'est la présence de certaines méthodes ou attributs qui définira ce qu'il est.
- ▶ Le *duck typing* donne de l'importance à ce que peut faire un objet plutôt qu'à ce qu'il est.
 - ▶ Ce qui est important sont les méthodes (comportement) et non son type.



Introduction

- ▶ Le terme *duck typing* vient d'une phrase très connue qui dit à peu près :
 - ▶ *"If it swims like a duck and quacks like a duck, then it probably is a duck".*
- ▶ Ce concept est complètement intégré dans Python qui utilise des mots tels que **sequence**, **iterable** ou encore **callable** pour définir le comportement d'un objet plutôt que son type.
- ▶ Ces termes, orientés comportement, montrent bien l'intégration du *duck typing* dans Python.
 - ▶ L'important est le comportement, pas l'état.



Duck typing : Exemple basic

- ▶ La fonction `len()` permet de récupérer la taille d'un élément.
- ▶ Mais que fait exactement `len()` ?
- ▶ Est-ce que tous les objets ont une taille ?
- ▶ En fait `len()` appelle la méthode `__len__()` de l'objet.
 - ▶ Ainsi il suffit de l'implémenter pour qu'un objet ait une taille.

```
class ImVeryLong:
    def __len__(self):
        return 12000

a = ImVeryLong()
print(len(a))
12000
```

- ▶ Les entiers n'ont pas de taille, les dictionnaires oui.
 - ▶ Ce qui est visible dans les sorties de `dir(int)` et `dir(dict)`.



Duck typing : Fichier

- Un fichier est simplement un objet qui possède les méthodes `read()`, `write()` et `close()`.
- Et tout objet qui respecte cette définition sera considéré comme un fichier.

```
class MyFile:
    def __init__(self):
        self.data=""
    def read(self, size=0):
        return self.data[0:size]
    def write(self, s):
        self.data += s
        return 0
    def close(self):
        pass

mf = MyFile()
print("Hello Duck", file=mf)
print(mf.read(8))
```



Duck typing : Context manager

- ▶ Un *context manager* est un objet Python qui est appelé avec l'instruction **with** :

```
with context_manager_object:  
    # Do something clever  
pass
```

- ▶ L'entrée dans le block **with**, appelle la méthode **__enter__**.
- ▶ Lors de la sortie c'est la méthode **__exit__** qui est appelée.
 - ▶ **__exit__** est **toujours appelée**, même si une exception se produit lors de l'exécution (identique à un **finally**).
 - ▶ Si une exception se produit elle sera **transmise**.
- ▶ Un *context manager* est simplement un objet qui implémente ces deux méthodes.
 - ▶ On parle du **context manager protocole**.



Duck typing : Context manager

► Par exemple :

```
class Hello:
    def __init__(self, name):
        self.name = name
    def __enter__(self):
        print(f"Hello {self.name}!")
        return self
    def __exit__(self, ex_type, ex_value, ex_traceback):
        print(f"Goodbye {self.name}")

with Hello("World"):
    print("Doing some clever stuff...")

Hello World!
Doing some clever stuff...
Goodbye World
```



Duck typing : Context manager

- ▶ Généralement un *context manager* retourne un objet avec lequel on effectue des manipulations.
- ▶ L'objets est alors récupéré avec le mot clé **as** :

```
with open(filename) as fd:  
    pass
```

- ▶ Rappel :
 - ▶ Lors de l'utilisation d'un *context manager*, si une exception se produit, la méthode **__exit__** sera appelée et l'exception sera transmise au code appelant.
 - ▶ Ainsi lors de l'utilisation d'un contexte manager pour manipuler un fichier, même si une exception se produit, le fichier sera de toute manière fermé.



Duck typing : Context manager

- ▶ Un *context manager* est utile dans les cas suivants :
 - ▶ **Open - Close**
Ouvrir et fermer automatiquement une ressource, telle que :
 - Manipulation de fichiers.
 - Connexions de base de données.
 - Connexions réseau.
 - ▶ **Lock - Release**
Verrouiller et libérer une ressource.
 - ▶ **Start - Stop**
Dans le cas de scénario nécessitant le démarrage et l'arrêt automatique d'un processus.
 - Par exemple dans le cas d'un *timer*.
 - ▶ **Change - Reset**
Dans le cas où il faut effectuer une modification d'une configuration, d'une connexion puis revenir à la précédente.



Duck typing : Itérable

- ▶ La logique *duck typing* s'applique également à tous les objets de type itérable.
- ▶ En effet, un itérable est simplement un objet qui fournit les méthodes `__iter__` et `__next__`.
 - ▶ Comme on le verra plus tard.



Exercices 2

- Créez un *context manager* qui délimite le code exécuté avec un nombre de caractères défini par ses paramètres. Par exemple :

```
# The following call
with CodeDelimiter("=", 50):
    odd_max = 10
    print("Odd numbers lower then {odd_max}:", end=' ')
    print(extract_odd(odd_max))

# Produce this output
=====
Odd numbers lower then 20: [1, 3, 5, 7, 9]
=====
```

Programmation fonctionnelle

Lambda, filter, map, reduce

List comprehensions

Itérateurs

Générateurs



Fonction anonyme : Expressions lambda

- ▶ Une fonction anonyme est une fonction qui n'a pas de nom.
- ▶ En Python la création d'une fonction classique est réalisée avec le mot clé **def**.
- ▶ Une fonction anonyme sera créée avec le mot clé **lambda**.
- ▶ Une fonction anonyme est donc également appelée **fonction lambda** ou **expression lambda**.
- ▶ Ces fonctions sont constituées d'une liste d'arguments et d'une seule expression qui est retournée par le lambda :

```
lambda arguments: expression
```

- ▶ Que l'on pourra utiliser pour créer simplement une fonction :

```
sum = lambda a, b: a + b  
sum(3, 4)  
7
```



Fonction anonyme : Expressions lambda

- ▶ Une expression lambda devient vraiment intéressante lorsqu'elle est utilisée dans une fonction paramétrée :

```
def power_generator(n):  
    return lambda x: x**n  
  
power_2 = power_generator(2)  
power_5 = power_generator(5)  
power_2(2)  
4  
power_5(2)  
32
```

- ▶ Un lambda est généralement utilisé en combinaison avec d'autres fonctions de plus haut niveau.
- ▶ Typiquement lorsque la fonction prend une fonction en argument.
 - ▶ Par exemple avec les fonctions `filter()`, `map()` et `reduce()`.



`filter()`, `map()` et `reduce()`

- ▶ Ces trois fonctions prennent en paramètre une fonction et un objet itérable.
- ▶ `map()` applique la fonction à chaque élément de l'objet.
 - ▶ La fonction à appliquer n'a donc qu'un seul paramètre.
- ▶ `filter()` filtre les éléments en fonction du retour booléen de la fonction.
 - ▶ La fonction à appliquer n'a donc également qu'un seul paramètre.
- ▶ Ces deux fonctions retournent un **itérateur**.
- ▶ `reduce()` est basé sur le fait que l'on peut souvent réduire un problème à une opération ou à une suite d'opérations sur deux éléments.
- ▶ `reduce()` fonctionne de la manière suivante :
 - ▶ Il prend les deux premiers éléments de la séquence et effectue la fonction sur ces deux éléments.
 - ▶ Il conserve le résultat et prend l'élément suivant de la séquence et applique la fonction sur le résultat précédent et ce nouvel élément.
 - ▶ Et ainsi de suite...
 - ▶ La fonction à appliquer a donc **deux** paramètres.
 - ▶ `reduce()` permet ainsi de réduire une séquence en un simple résultat.



`filter()` et `map()` et `reduce()`

- Pour tripler les valeurs d'une liste puis ne conserver que les valeurs paires, on pourrait faire la chose suivante :

```
l = list(range(0, 10))  
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]  
  
l = list(map(lambda x: x*3, l))  
[0, 3, 6, 9, 12, 15, 18, 21, 24, 27]  
  
l = list(filter(lambda x: x % 2 == 0, l))  
[0, 6, 12, 18, 24]
```



`filter()` et `map()` et `reduce()`

- Pour profiter des caractéristiques des itérateurs à tous les niveaux l'exécution des instructions précédentes serait réalisée en une seule instruction :

```
l = list(filter(lambda x: x % 2 == 0,  
               map(lambda x: x*3, range(10))))  
[0, 6, 12, 18, 24]
```

- Ou pour trouver la valeur maximale d'une liste mixte :

```
from functools import reduce  
l = ["12", 10, "23.12", 1.5, 15, "32"]  
res = reduce(lambda x, y: max(float(x), float(y)), l)  
32.0
```



List comprehension

- ▶ Van Rossum n'est pas fan des **lambda**, **filter**, **map** et autres **reduce**.
- ▶ Il conseille d'utiliser la syntaxe des **list comprehension** ou des boucles **for** à la place car ces deux dernières approches sont plus lisibles et mieux compréhensibles.
- ▶ Une **list comprehension** est une syntaxe qui permet de créer une liste à partir d'une autre liste :

```
[expression for item in list]
```

- ▶ Pour créer une liste des lettres d'un mot :

```
name = "Philippe"  
[c for c in name]  
['P', 'h', 'i', 'l', 'i', 'p', 'p', 'e']  
set([c for c in name])  
{'P', 'l', 'e', 'i', 'p', 'h'}
```



List comprehension : Conditions **if**

- La syntaxe des *list comprehensions* supporte l'ajout de conditions :

```
[expression for item in list if condition]
```

```
[x for x in range(100) if x % 2 == 0 and x % 5 == 0]  
[0, 10, 20, 30, 40, 50, 60, 70, 80, 90]
```

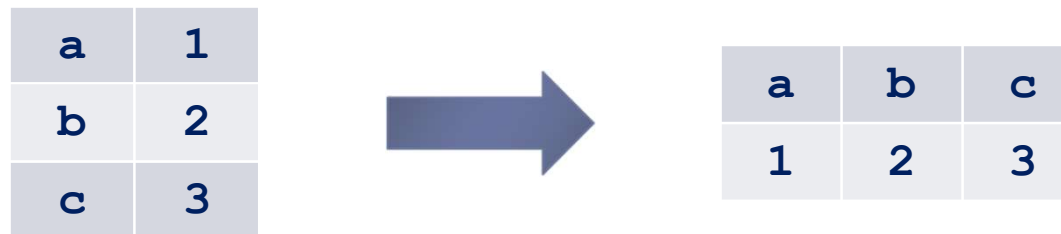
- La première partie étant une expression, il est tout à fait possible d'utiliser un **if** ternaire, ce qui donne :

```
["Pair" if x % 2 == 0 else "Impair" for x in range(4)]  
['Pair', 'Impair', 'Pair', 'Impair']
```



List comprehension : Conditions **if**

- ▶ Il est même possible d'imbriquer les *list comprehensions*.
- ▶ Par exemple pour transposer une matrice :



```
data = [['a', 1], ['b', 2], ['c', 3]]  
[[row[i] for row in data] for i in range(2)]  
[['a', 'b', 'c'], [1, 2, 3]]
```

- ▶ Mais pour effectuer une telle opération on utilise **pandas**.
Il s'agit d'un module Python dédié à l'analyse de données.



Dict comprehension

- ▶ La logique des *list comprehensions* est également utilisable avec des dictionnaires.
- ▶ Avec une syntaxe un peu différente :

```
{new_key:new_value for (key, value) in dict.items()}
```

- ▶ Par exemple pour modifier les prix d'une liste de prix :

```
price = {'Xiaomi': 200, 'Samsung': 700, 'iPhone': 1500}  
new_price = {key:value*0.9 for (key, value) in prices.items()}  
{'Xiaomi': 180.0, 'Samsung': 630.0, 'iPhone': 1350.0}
```

- ▶ Ou associer une réduction aléatoire à des clients :

```
from random import randint  
customers = ["Julien", "Prunille", "Pierre", "Paul"]  
discount = {customer:randint(5, 20) for customer in customers}  
{'Julien': 16, 'Prunille': 13, 'Pierre': 19, 'Paul': 12}
```



Dict comprehension

- ▶ La syntaxe *dict comprehension* supporte également l'ajout de condition et l'utilisation du **if** ternaire.

```
{new_key:new_value for (key, value)  
in dict.items() if condition}
```

- ▶ L'une des manipulations pour laquelle la syntaxe *dict comprehension* est la plus utilisée et celle permettant d'associer deux listes disjointes en un dictionnaire.
 - ▶ Pour cela la fonction **zip()**, interne à Python, est très utile.

```
month = ["Jan", "Feb", "Mar", "Apr", "May", "Jun"]  
income = [12455, 16211, 13503, 17840, 18120, 14200]  
control = {m:i for (m, i) in zip(month, income)}  
{ 'Jan': 12455, 'Feb': 16211, 'Mar': 13503, ... }
```



Set comprehension

- Finalement la logique des *list comprehensions* est également utilisable avec les **set** :

```
{expression for item in iterable}
```

```
name = "Philippe"  
{c for c in name}  
{ 'P', 'l', 'e', 'i', 'p', 'h' }
```



Exercices 3

- ▶ Écrivez un programme qui analyse une liste et produit 2 nouvelles listes contenant les nombres pairs pour l'une et impairs pour l'autre.
- ▶ En utilisant le principe du Crible d'Eratosthène affichez les nombres premiers plus petits que 100.



Itérateurs

- ▶ Un itérateur est un objet que l'on peut itérer.
 - ▶ C'est-à-dire un objet représentant un flux de données et capable de les retourner l'une après l'autre.
- ▶ Le concept d'itérateurs est très présent dans Python, on le retrouve derrière la boucle **for**, les *list comprehensions*, les générateurs, etc.
- ▶ Au niveau de l'implémentation, un itérateur est une classe qui implémente les méthodes `__iter__()` et `__next__()`, appelé le protocole des itérateurs (*iterator protocol*).
- ▶ Un objet duquel il est possible de récupérer un itérateur est dit **itérable**.
- ▶ La fonction `iter()` permet de récupérer un itérateur sur un objet.
 - ▶ Elle appelle la méthode `__iter__()` de l'objet.
- ▶ La plupart des conteneurs Python (**str**, **list**, **tuple**, ...) sont itérables.
- ▶ Ensuite la fonction `next()` est utilisée pour itérer un itérateur.
 - ▶ Elle appelle la méthode `__next__()` de l'objet.



Itérateurs

- ▶ Lorsqu'il n'y a plus d'élément à itérer l'exception **StopIteration** est envoyée.
 - ▶ Ce qui est géré automatiquement par la boucle **for**.
- ▶ En cas d'appels explicites de la fonction **next()**, il faudra traiter l'exception :

```
l = ["Pierre", "Paul"]
i = iter(l)
user = next(i)
'Pierre'
user = i.__next__()
'Paul'
user = next(i)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
StopIteration
```

- ▶ L'utilisation d'une boucle **for** est la bonne approche pour parcourir un itérateur.
- ▶ L'autre manière serait d'utiliser un boucle **while** avec un **break** sur l'exception.
 - ▶ Ce qui correspond à la manière dont est implémentée la boucle **for**.



Création d'un itérateur

- ▶ Pour définir un itérateur il suffit d'implémenter les méthodes nécessaires.
- ▶ Si la classe contient la méthode `__next__()` elle sera son propre itérateur, la méthode `__iter__()` pourra donc simplement retourner `self`.
- ▶ Il est également nécessaire d'implémenter l'envoi de l'exception `StopIteration` à la fin de l'itération.
- ▶ Pour créer un itérateur infini il suffit de ne pas lancer d'exception et de toujours retourner une valeur.
 - ▶ Il faudra juste faire attention si un tel itérateur est utilisé dans une boucle `for`...



Création d'un itérateur

- Exemple d'un itérateur qui retourne la racine carrée de nombres jusqu'à un maximum défini :

```
class SquareRoot:
    def __init__(self, max = 0):
        self.max = max

    def __iter__(self):
        self.cur = 0
        return self

    def __next__(self):
        if self.cur <= self.max:
            res = self.cur ** (1/2)
            self.cur += 1
            return f"{res:.3f}"
        else:
            raise StopIteration
```



Générateurs

- ▶ Le fonctionnement classique d'une fonction est de créer un espace de noms privé dans lequel sont créées les variables de la fonction. Lorsque le mot clé **return** est rencontré, une valeur est retournée et les variables sont détruites.
- ▶ Ce qui serait intéressant serait de pouvoir construire une fonction qui **conserve** son état entre les différents appels.
- ▶ C'est ce que permet le mot clé **yield** utilisé à la place de **return** dans les générateurs.
- ▶ Lorsqu'une fonction contient le mot clé **yield** elle sera automatiquement considérée comme un générateur et Python la compilera de manière spéciale.
- ▶ Un générateur ne retourne pas une valeur mais retourne un objet de type **generator** qui support l'*iterator protocol*.



Générateurs

- ▶ La grande différence entre **return** et **yield** est que, lors de l'exécution de **yield**, l'état de la fonction est **suspendu** et les variables locales sont conservées.
- ▶ Au prochain appel du générateur avec la méthode **next()**, la fonction reprendra son exécution **là où elle s'était arrêtée**.

```
def city_generator():  
    yield "Lausanne"  
    yield "Genève"  
  
city = city_generator()  
print(next(city))  
Lausanne  
print(next(city))  
Genève  
print(next(city))  
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
StopIteration
```



Générateurs

- ▶ La définition d'un itérateur nécessite d'implémenter les méthodes `__iter__()` et `__next__()`, conserver l'état interne des variables et lancer `StopIteration` lorsqu'il n'y a plus de valeur à retourner.
- ▶ Les générateurs représentent une manière bien plus simple de créer des itérateurs sans devoir se soucier de tout ce qui est énuméré au-dessus.
- ▶ Et de manière identique à un itérateur, un générateur peut être utilisé directement dans une boucle `for` :

```
for i in city_generator():  
    print(i)  
Lausanne  
Genève
```



Générateurs

- ▶ En utilisant le concept de générateur, le code nécessaire à l'implémentation de la logique de la classe **SquareRoot** précédente est bien plus simple :

```
def square_root(max = 0):  
    cur = 0  
    while cur <= max:  
        yield cur ** (1/2)  
        cur += 1
```

- ▶ Que l'on utilisera simplement de cette manière :

```
for i in square_root(3):  
    print(i)
```



Envoyer des valeurs au générateur

- ▶ Les générateurs ne sont pas simplement capables de produire des données, ils peuvent également en recevoir.
- ▶ Pour réaliser cela l'envoi d'un message au générateur est effectué à l'aide de la méthode `send()` :

```
i.send(3)
```

- ▶ A l'intérieur du générateur le message sera récupéré par la même instruction `yield` de la manière suivante :

```
message = (yield cur ** (1/2))
```

- ▶ Les parenthèses autour de l'expression `yield` sont conseillées car elles sont parfois nécessaires (PEP 342).



Envoyer des valeurs au générateur

- ▶ Lors de l'envoi d'un message à un générateur il faut tenir compte des deux aspects suivants :
 - ▶ Un générateur ne peut pas recevoir de message tant qu'il n'a pas été initialisé.
 - Cela signifie qu'un premier appel à `next()` est nécessaire pour atteindre l'instruction `yield`.
 - ▶ Lors de l'envoi d'un message il se produit 2 choses :
 1. Le message est envoyé au générateur.
 2. Le générateur retourne une valeur.
 - ▶ Si cela n'est pas pris en compte, l'envoi d'une valeur au générateur **fera avancer le générateur** d'un élément.



Envoyer des valeurs au générateur

- ▶ Sachant que lors du parcours du générateur en lecture, soit avec l'instruction `next()`, la valeur renvoyée par le générateur est `None`.
- ▶ La logique suivante sera suffisante :

```
message = (yield cur ** (1/2))
if message is None:
    # NO message received
    # Increment the current position
else:
    # We got a message !
    # Do something with the message
```



Generator expression

- ▶ Tout comme un lambda permet de créer une fonction anonyme, il est possible de créer très simplement un générateur anonyme.
- ▶ La syntaxe est identique à celle des *list comprehensions* mais en utilisant des parenthèses à la place des crochets :

```
l = [1, 2, 3]                # Initialize a list
l_pow = [x**2 for x in l]    # Initialize another list
g_pow = (x**2 for x in l)    # Initialize a generator
```

- ▶ Le grand intérêt de cela est plus visible lorsque les listes utilisées sont très grandes et prennent donc beaucoup de mémoire lorsqu'elles sont initialisées en tant que listes.



Generator expression

- ▶ Dans l'exemple précédent, pour être encore plus efficace, la bonne approche serait d'initialiser la première liste avec un générateur :

```
l = range(1, 4)
```

- ▶ Ou d'initialiser la 2ème liste en une seule instruction :

```
g_pow = (x**2 for x in range(1, 3))
```



Intérêt des générateurs

- ▶ Les générateurs offrent une implémentation plus propre et plus concise pour les itérateurs.
- ▶ Ils permettent de consommer beaucoup moins de mémoire lors du traitement d'éléments itérables.
- ▶ Ils permettent de représenter des séquences infinies qu'il n'est pas possible de stocker en mémoire.
- ▶ Ils peuvent être chaînés pour exécuter une suite d'opérations :

```
g = (x**2 for x in range(1,20))  
p = (x for x in g if x%2 == 0)  
  
for i in p:  
    print(i)
```

- ▶ Ce qui est très intéressant au niveau consommation mémoire.



Exercices 4

- ▶ Créez un générateur qui inverse un itérable (**string**, **list**, ...) qui lui est passée en argument.
 - ▶ Testez avec des chaînes de caractères pour commencer.
- ▶ Créez un générateur qui retourne la puissance **N** des premiers **X** nombres entiers.
 - ▶ **X** est passé au moment de la création du générateur.
 - ▶ **N** vaut **1/2** par défaut, mais peut être modifiée en cours d'utilisation.
- ▶ Soit le fichier de log d'une chaîne de pizza (**ventes.log**).
 - ▶ La **4^{ème}** colonne de chaque ligne affiche le nombre de pizzas vendues chaque heure ou **N/A** si aucune pizza n'a été vendue.
 - ▶ Vous vous intéressez au total des pizzas vendues ces **10** dernières années.
 - ▶ Considérez que le fichier contient exactement les **10** dernières années.



Manipulations de dictionnaires



Introduction

- ▶ En Python 2.7 les dictionnaires étaient des structures non ordonnées mais tout de même déterministes.
 - ▶ Cela signifie que les éléments n'étaient pas forcément retournés dans l'ordre dans lequel ils avaient été enregistrés.
 - ▶ Mais qu'ils étaient toujours retournés dans le même ordre.
- ▶ Avec Python 3.0 les dictionnaires sont devenus aléatoires.
 - ▶ Les éléments n'étaient pas forcément retournés dans l'ordre dans lequel ils avaient été enregistrés mais en plus cet ordre pouvait être différent entre différents appels.
- ▶ Depuis Python 3.6 les dictionnaires sont à nouveau des structures ordonnées.
 - ▶ Les éléments sont retournés dans l'ordre dans lequel ils ont été enregistrés.



Itération sur un dictionnaire

- ▶ Avec une boucle **for**, l'itération s'effectue automatiquement sur les clés du dictionnaire.
- ▶ Pour forcer ou modifier ce comportement on utilise les méthodes **keys()** ou **values()** afin de retourner la liste des clés ou des valeurs et itérer sur cette liste.
- ▶ La méthode **items()** retourne une liste de tuples de deux éléments correspondant à chaque élément du dictionnaire.
 - ▶ Cette approche permet de récupérer tant la clé que la valeur en une seule instruction :

```
julien = {'first': "Julien", 'name': "Ambord"}  
for key, value in julien.items():  
    print(f"{key} => {value}")  
first => Julien  
name => Ambord
```



Limiter les éléments

- ▶ Il peut être intéressant de récupérer uniquement certaines informations d'un dictionnaire, mais pas toutes.
- ▶ La méthode **keys()** retourne les clés du dictionnaire.
 - ▶ Celles-ci sont uniques, comme un **set**.
- ▶ Ce qui permet d'effectuer des opérations de la théorie des ensembles, comme pour les sets :

```
julien = {'first': "Julien", 'name': "Ambord",  
         'phone': "021 432 56 78", 'email': "julien@yop.com"}  
  
for key in julien.keys() - {'phone', 'email'}:  
    print(f"{key} => {julien[key]}")  
first => Julien  
name => Ambord
```



Trier un dictionnaire

- ▶ Les dictionnaires sont des structures ordonnées.
- ▶ Il est donc possible de trier un dictionnaire.
- ▶ Pour cela on utilise la fonction `sorted()`, interne à Python, associée à la syntaxe *dict comprehension* :
 - ▶ `sorted()` agira par défaut sur les clés du dictionnaire.

```
data = {'Julien': 8, 'Anthéa': 9, 'Prunille': 7}
data = {key:data[key] for key in sorted(data)}
data
{'Anthéa': 9, 'Julien': 8, 'Prunille': 7}
```

- ▶ Pour trier selon les valeurs on utilise le 2ème paramètre de `sorted()` qui permet de fournir une fonction de tri selon la même logique que la méthode `sort()` des `list`.



Combiner des dictionnaires

- ▶ L'opérateur `+` ne permet pas de combiner 2 dictionnaires.
- ▶ Il existe cependant différentes manières de le faire :
 - ▶ Avec une boucle en ajoutant les éléments du 2^{ème} dans le premier.
 - Un des dictionnaires est modifié.
 - ▶ En utilisant la méthode `update()` de la classe `dict`.
 - Un des dictionnaires est modifié.
 - ▶ En utilisant la fonction `ChainMap()` du module `collections`.
 - Un nouveau dictionnaire est créé.
 - ▶ En utilisant l'opérateur de déballage `**`.
 - Un nouveau dictionnaire est créé.
 - ▶ En utilisant l'opérateur `|` depuis Python 3.9, ou son raccourci `|=` qui permettent de combiner des dictionnaires.
 - Un nouveau dictionnaire est créé ou un des dictionnaires sera modifié.



Combiner des dictionnaires

```
for key, value in d2.items():  
    d1[key] = value
```

```
d1.update(d2)
```

```
from collections import ChainMap  
d3 = ChainMap(d1, d2)
```

```
d4 = {**d1, **d2}
```

```
d5 = d1 | d2  
d1 |= d2
```

- L'approche la plus rapide est celle utilisant **ChainMap**.
 - **ChainMap** semble avoir une complexité **O(1)**, alors que les autres dépendent du nombre d'éléments à ajouter.



Typen le code Python



Introduction

- ▶ Python étant typé dynamiquement, une variable peut contenir n'importe quel type de données.
 - ▶ Et surtout, son type peut changer en cour d'exécution
- ▶ Ceci offre une grande flexibilité, mais peut également rendre le code moins sûr.
- ▶ Soit la fonction suivante :

```
def double_it(a):  
    return a * 2
```

- ▶ Il semble évident qu'elle a été définie pour être utilisée dans un contexte numérique.



Introduction

- ▶ Elle pourrait cependant être utilisée de cette manière :

```
z = "3"  
y = 10 + double_it(z)
```

- ▶ Ce qui produit un erreur de type (**TypeError**).
- ▶ En typant le paramètre **a** comme un **int** ou un **float**, l'erreur aurait été détectée plus tôt.
- ▶ Et lors de la définition de tests, le typage permettra d'éviter de devoir définir des tests relatifs à la simple validation des données.



Introduction

- ▶ Un autre aspect intéressant du typage concerne la complétion de code dans les IDEs.
- ▶ Soit la fonction suivante :

```
def full_name(first, name):  
    return f"{first.title()} {name.title()}"
```

- ▶ Lors de l'écriture du corps de la fonction la question qui se pose sera de savoir comment s'appelle la méthode qui permet de mettre en majuscule la première lettre des mots ?
 - ▶ Est-ce `upper_first_char` ? `capitalize` ? `word_title` ? ...
- ▶ L'idée sera alors d'utiliser l'auto-complétion de l'IDE.



Introduction

- ▶ On écrit **first**, puis le point (.) puis **[CTRL]+Space** pour afficher les complétions.
- ▶ Mais ça ne fonctionne pas.
 - ▶ L'IDE ne peut pas savoir de quels types sont les paramètres.
 - ▶ Il ne peut donc proposer aucune complétion.
- ▶ Si les paramètres avaient été typés, l'IDE aurait été capable d'afficher la liste des attributs et méthodes en relation avec le type de chaque paramètre.



Introduction

- ▶ Le typage des variables permet donc :
 - ▶ D'écrire du code plus sûr.
 - ▶ D'écrire moins de tests.
 - ▶ De simplifier le travail du développeur.
- ▶ Python ne peut pas être typé de manière magique, mais il est possible d'ajouter des informations sur certains éléments du langage.
 - ▶ Il s'agit des **annotations**.
- ▶ Ces informations pourront être prises en compte par des outils tiers tels que des vérificateurs de types, des IDE, des linters, etc.



Annotations

- ▶ Les annotations sont apparues avec Python 3.
- ▶ L'idée était de pouvoir associer des labels :
 - ▶ Aux variables.
 - ▶ Aux attributs d'une classe.
 - ▶ Aux paramètres d'une fonction.
 - ▶ Aux valeurs de retour des fonctions.
- ▶ Ces annotations sont conservées et accessibles par l'attribut **__annotations__**.
 - ▶ Seules les fonctions possèdent cependant ce paramètre.
- ▶ Depuis Python 3.5 les annotations sont utilisées pour ajouter les informations de type sur les variables et les fonctions :
 - ▶ Types des variables.
 - ▶ Types des paramètres des fonctions et type de retour des fonctions.



Typage

- ▶ Le typage est fournit par le module **typing**.
- ▶ Avec les dernières versions de Python (3.9) il n'est plus nécessaire de le charger.
- ▶ La syntaxe pour le typage d'une fonction est la suivante :
 - ▶ Le typage des paramètres utilise le ":".
 - ▶ Si le paramètre possède une valeur par défaut, celle-ci est définie avec le signe **=**.
 - ▶ La définition du type de retour se fait avec une flèche (**->**).
- ▶ La syntaxe est très aérée :

```
def do_something(a: int = 0, b: int = 0, c: int = 0) -> int:  
    return (a + b) * c
```



Typage

- ▶ Les types qui peuvent être utilisés sont les types de bases :
 - ▶ `int`, `float`, `bool`, `str`, `bytes`.
- ▶ Il est également possible d'utiliser des types plus complexes, tels que les listes.
- ▶ Le module `typing` fournit des types spéciaux, par exemple :
 - ▶ Pour les conteneurs classiques : `List`, `Dict`, `Tuples`, `Set`.
 - ▶ Pour créer des alternatives : `Union`.
- ▶ Depuis Python 3.10 l'utilisation de ces éléments n'est pas nécessaire.
 - ▶ Les types Python peuvent être utilisés directement.



Typage

- ▶ Ainsi pour préciser qu'un paramètre est une liste on utilise... **list** :

```
def sum_it(l: list) -> int:  
    return sum(l)
```

- ▶ Dans ce cas ce serait bien de dire que la liste ne doit contenir que des nombres entiers.
- ▶ Ce que l'on fait en précisant le type des données de la liste à l'aide de la notation **[]** :

```
def sum_it(l: list[int]) -> int:
```



Typage

- L'opérateur `|` permet de définir des alternatives :

```
def double_it(l: int | str) -> int | str:  
    return sum(l)
```

- La définition d'un paramètre optionnel s'effectue à l'aide d'une alternative avec `None` et d'une valeur par défaut :

```
def mul_it(a: int|str, nb: int | None = 1) -> int|str:  
    return a if not nb else a * nb
```



dict, set et tuple

- ▶ Le typage des **set** est identique aux **list**.
- ▶ Pour les dictionnaires, il est possible de spécifier les types des clés et des valeurs :

```
def print_info(info: dict[str, str]) -> None:  
    for k, v in info.items():  
        print(f"{k} => {v}")
```

- ▶ Pour les tuples de taille fixe il est possible de spécifier chaque élément du tuple :

```
def print_tuple(t: tuple[str, str]) -> None:  
    print(t)
```



dict, set et tuple

- ▶ Sinon on utilise l'ellipse (...) pour signifier que le même type se répète :

```
def print_tuple(t: tuple[int, ...]) -> None:  
    print(t)
```

- ▶ Finalement les définitions de types peuvent également être utilisées avec les classes personnelles :

```
class Pet:  
    def __init__(self, name: str) -> None:  
        self.name = name  
  
def free_pet(p: Pet):  
    print(f"{p.name}: I'm not a number, I'm a free Pet!")
```



Variables

- Les définitions de types peuvent évidemment s'appliquer également aux variables :

```
i: int = 1
f: float = 1.0
b: bool = True
s: str = "test"
b: bytes = b"test"

l: list[int] = [1, 2, 10]
l: list[int | str] = [3, 5, "test", "fun"]
d: dict[str, float] = {"field": 2.0}
s: set[int] = {6, 7}
t: tuple[int, str, float] = (3, "yes", 7.5)
t: tuple[int, ...] = (1, 2, 3)
```



Vérifier les types

- ▶ La package le plus utilisé est **mypy**.
 - ▶ C'est d'ailleurs lui qui est à l'origine de l'intégration des types dans Python.
 - ▶ PyCharm utilise sa propre implémentation.
 - ▶ Dans VSCode, l'extension **Pylance** utilise **pyright** de MS.

- ▶ Vérification des types pour un fichier ou un répertoire :

```
[pny]$ python -m mypy file.py  
[pny]$ python -m mypy project/
```

- ▶ **Mypy** fera le nécessaire de manière récursive sur tous les fichiers Python.



Vérifier les types à l'exécution

- ▶ **Mypy** permet d'effectuer des vérifications de types de manière statique.
- ▶ Ce qui n'est parfois pas suffisant.
 - ▶ Typiquement lorsque les données proviennent de sources externes.
 - Entrées des utilisateurs ou API web.
- ▶ Dans ces cas on utilise des vérificateurs de type en cours d'exécution.
- ▶ **Pydantic** est l'un de ces outils.
 - ▶ Il effectue des conversions de types afin que les données se conforment aux types attendus.
- ▶ **Pydantic** est, par exemple, intégré à **FastAPI**.



Vérifier les types à l'exécution

- ▶ **Marshmallow** est un outil semblable à **Pydantic**.
- ▶ L'utilisation de tels outils assure que les données correspondent aux types définis.
 - ▶ Il en résulte moins de bogues et moins de tests à écrire.
- ▶ **Typeguard** est également un outil permettant la validation de types à l'exécution.
 - ▶ **Mypy** effectue une vérification statiques des types.
 - ▶ **Pydantic** et **Marshmallow** se concentre sur la validation de données.
 - ▶ **Typeguard** effectue une vérification des types au moment de l'exécution des fonctions.



Modules et packages



Définition

- ▶ Une **fonction** est un bloc de code que l'on peut réutiliser en l'appelant par un mot clé constitué de son nom.
 - ▶ Par exemple la fonction `print()`.
- ▶ Un **module** est un fichier Python (`.py`) qui contient une liste de fonctions, de variables, de classes.
 - ▶ Par exemple dans `math.pi`, `pi` est une variable du module `math`.
 - ▶ Dans `time.sleep()`, `sleep()` est une fonction du module `time`.
- ▶ Un **package** est une collection de modules Python.
 - ▶ Par exemple dans `numpy.random.randint()`, `randint()` est une fonction du module `random` du package `numpy`.
- ▶ Certains modules sont directement intégrés à Python alors que d'autres peuvent être installés séparément.
- ▶ Un module étant un simple fichier Python, chaque développeur peut très facilement écrire ses propres modules.
- ▶ Le nom d'un module ou d'un package représente un espace de noms.



Utilisation d'un module

- ▶ Pour pouvoir utiliser les éléments définis dans un module, il faut importer le module dans le programme courant.
- ▶ Ce qui se fait avec le mot clé **import** :

```
import time  
print(time.asctime())
```

- ▶ Selon la PEP 8 l'import de plusieurs modules se fait sur plusieurs lignes :

```
import os  
import sys  
import time
```



Utilisation d'un module

- ▶ Dans le cas de l'import du module complet, le nom du module est nécessaire pour accéder aux éléments qu'il contient.
- ▶ Pour pouvoir accéder directement un élément présent dans un module, il faut importer cet élément (**import**) depuis (**from**) le module :
 - ▶ Pour importer tous les éléments (sauf ceux commençant par un **_**) on utilise l'astérisque *****.

```
from time import asctime, localtime  
from random import random  
from re import *
```

- ▶ L'import complet des éléments est déconseillé car :
 - ▶ Cela augmente les risques de collisions.
 - ▶ Ce n'est pas très efficace si le nombre d'éléments à importer est grand par rapport au nombre d'éléments utilisés.
 - ▶ On perd l'information de provenance des éléments.



Alias

- Il est possible de modifier le nom du module ou des symboles importés avec le mot clé **as** :

```
import pandas as pd  
import numpy as np  
from numpy import random as rnd
```



Modules utiles : **logging**

- ▶ Très utile pour savoir ce qui se passe dans le programme.
 - ▶ En particulier pour les programmes qui s'exécutent en tâche de fond (service/daemon) ou pour les gros programmes.
- ▶ Le module fournit des fonctions permettant de *logger* des messages selon 5 niveaux de sévérité :

Niveau	Méthode associée
DEBUG	<code>logging.debug()</code>
INFO	<code>logging.info()</code>
WARNING	<code>logging.warning()</code>
ERROR	<code>logging.error()</code>
CRITICAL	<code>logging.critical()</code>



Modules utiles : **logging**

```
import logging
logging.warning("Warning message")
WARNING:root:Warning message
```

- ▶ L'affichage montre la sévérité du message, le nom du *logger* par défaut (**root**) et le message.
- ▶ Par défaut la sévérité minimale générant un enregistrement est configurée à **WARNING** et la sortie se fait sur la console.
- ▶ Ce qui signifie que :
 - ▶ Seuls les messages de sévérité **WARNING** et plus grave seront *loggués*.
 - ▶ L'affichage des messages de log est identique à un **print()**.
 - Sauf que la sortie se fait sur **STDERR** par défaut, redirigée sur la console.



Modules utiles : **logging**

- ▶ Pour que les messages soient envoyés dans un fichier il suffit de configurer le *logger*.
- ▶ Ce qui se fait à l'aide de la fonction **basicConfig()** à laquelle on passe généralement des attributs pour les paramètres **filename** et **level** :

```
logging.basicConfig(filename="debug.log", level=logging.DEBUG)
```

- ▶ Par défaut le fichier de log (**filename**) sera ouvert en mode '**a**' (append).
 - ▶ Pour écraser le fichier à chaque démarrage du programme on renseigne également le paramètre **filemode='w'**.



Modules utiles : **logging**

- ▶ Un autre paramètre important est le **format**, que l'on peut également définir à l'aide de **basicConfig()**.
- ▶ Par défaut le format est :

```
% (levelname) s : % (name) s : % (message) s
```

- ▶ Mais on peut le rendre plus intéressant à l'aide du paramètre **format** avec par exemple :

```
format="% (asctime) s [% (filename) s] % (levelname) s :  
% (message) s (Line: % (lineno) s) "
```

- ▶ Ce qui produira des messages tels que :

```
2021-10-09 08:59:51,944 [yop.py] WARNING: Mmmm... Something strange (Line:7)
```



Modules utiles : **logging**

- ▶ Logger la **stack trace**.
- ▶ Lorsqu'un message est *loggué* depuis une exception il est intéressant d'avoir également la trace de cette exception.
 - ▶ Pour cela il suffit d'ajouter **exc_info=True** lors du *logging* :

```
logging.error("An exception occurred", exc_info=True)
```

- ▶ Comme il est généralement toujours intelligent d'avoir la trace de la pile des appels lorsqu'on *loggue* sur exception, le module **logging** fournit une fonction prédéfinie :

```
logging.exception()
```

- ▶ *Loggue* au niveau **ERROR** avec **exc_info=True** par défaut.



Modules utiles : **logging**

- ▶ Dans le cas de gros programmes il est intéressant de créer ses propres *logger* en instanciant des objets de type **Logger**, **Handler** et **Formatter**, ce qui permettra de définir différents canaux et niveaux de *logging*.
- ▶ Prenez l'habitude d'intégrer la notion de *logging* dans vos développements 😊



Modules utiles : **re**

- ▶ Le module **re** fournit le support des expressions régulières :
 - ▶ **re.match()** Recherche un motif au début d'une chaîne
 - ▶ **re.search()** Recherche un motif dans une chaîne
Retourne la première occurrence
 - ▶ **re.findall()** Recherche toutes les occurrences du motif dans la chaîne
 - ▶ **re.split()** Découpe une chaîne en fonction d'un motif considéré comme séparateur
 - ▶ **re.sub()** Recherche la première occurrence d'un motif et le remplace par la chaîne de remplacement
 - ▶ **re.compile()** Compile une expression régulière
Permet d'améliorer les performances
- ▶ La compilation d'une expression régulière n'est utile que lorsque celle-ci est utilisée de nombreuses fois dans le programme, par exemple dans une boucle.



Modules utiles : **re**

- ▶ Lors d'une recherche, les fonctions **match()** et **search()** renvoient un objet de type **match** en cas de succès ou **None** si aucune correspondance n'est trouvée.
- ▶ Il est donc facile de tester le résultat avec **if** :

```
m = re.search(pattern, string)
if m:
    process(m)
```

- ▶ L'objet **match** retourné possède des méthodes permettant de récupérer des informations sur l'occurrence trouvée :
 - ▶ **match.group()** Retourne la correspondance trouvée
 - ▶ **match.start()** Retourne la position du début
 - ▶ **match.end()** Retourne la position de la fin
 - ▶ **match.span()** Retourne les 2 positions dans un tuple



Modules utiles : **re**

- ▶ Lorsque le motif de recherche définit des groupes à l'aide des parenthèses, l'objet **match** retourné contient les informations de chaque groupe :

- ▶ **match.groups()**

Retourne la correspondance trouvée pour chacun des groupes

- ▶ **match.group(group)**

Retourne la correspondance trouvée pour le groupe **group**

- ▶ **match.start(group)**

Retourne la position du début de la correspondance du groupe **group**

- ▶ **match.end(group)**

Retourne la position de la fin de la correspondance pour le groupe **group**.

- ▶ **match.span(group)**

Retourne les 2 positions dans un tuple



Modules utiles : **itertools**

- ▶ Le module **itertools** fournit des fonctions qui retournent des itérateurs complexes à l'aide d'itérateurs basics.
- ▶ Il est très utile pour construire différents types de séquences à l'aide d'itérateurs.
- ▶ Par exemple la fonction **zip** interne à Python permet de combiner plusieurs séquences, éléments par éléments, et fournit une liste de tuples pour chaque combinaison :

```
a = list(range(3))
b = ["a", "b", "c", "d"]
c = [{'first': "Julien"}, {'first': "Prunille"}]
res = zip(a, b, c)
list(res)
[(0, 'a', {'first': 'Julien'}),
 (1, 'b', {'first': 'Prunille'})]
```



Modules utiles : **itertools**

- ▶ **zip** n'est pas une fonction fournie par **itertools**, mais elle donne une idée de la logique de **itertools**.
- ▶ **zip** retourne un itérateur sur une séquence où chaque élément est un tuple contenant des correspondances pour chaque élément des séquences d'origine.
- ▶ Si l'on veut avoir une séquence ayant des éléments pour tous les éléments des listes d'origines on utilise alors la fonction **zip_longest** de **itertools** :

```
import itertools
res_long = itertools.zip_longest(a, b, c)
list(res_long)
[(0, 'a', {'first': 'Julien'}),
 (1, 'b', {'first': 'Prunille'}),
 (2, 'c', None), (None, 'd', None)]
```



Modules utiles : **itertools**

- ▶ Les méthodes de **itertools** les plus utilisées sont :
 - ▶ **chain()**, **count()** et **repeat()**.
- ▶ **itertools.chain(*iterables)**
 - ▶ Retourne un itérateur permettant de parcourir plusieurs séquences à la suite :

```
for i in itertools.chain(a, b):  
    print(i, end=' ')  
0 1 2 a b c d
```



Modules utiles : `itertools`

- ▶ `itertools.count(start=0, step=1)`
 - ▶ Retourne un itérateur sur une séquence infinie.
- ▶ L'intérêt de `count()` est que la valeur de `step` peut être négative, mais surtout qu'elle peut également être **réelle**.
 - ▶ Ce qui n'est pas le cas de la fonction `range()`.
- ▶ Le fait d'avoir une boucle infinie n'est pas forcément utile.
 - ▶ on peut le faire avec un `while`.
- ▶ Par contre `count()` devient intéressante lorsqu'elle est combinée avec une autre séquence, par exemple en utilisant `zip()` :

```
first = ["Julien", "Prunille"]
for i in zip(itertools.count(500,100), first):
    print(i)
(500, 'Julien')
(600, 'Prunille')
```



Modules utiles : **itertools**

► **itertools.repeat(object [,times])**

- Retourne, par défaut, un itérateur qui fournit le même objet à l'infini.
- Le paramètre **times** permet de limiter le nombre de répétitions.

```
for i in itertools.repeat(8, 3):  
    print(i, end=' ')  
8 8 8  
  
for i in itertools.repeat(first, 2):  
    print(i)  
['Julien', 'Prunille']  
['Julien', 'Prunille']
```



Modules utiles : **itertools**

- ▶ **itertools** fournit également des méthodes permettant de filtrer simplement des séquences :
 - ▶ `itertools.takewhile(predicate, iterable)`
 - ▶ `itertools.dropwhile(predicate, iterable)`
- ▶ Permettent de conserver ou enlever un élément de la séquence tant qu'une condition est vraie.
 - ▶ Dès que la condition devient fausse, le filtrage s'arrête.
 - ▶ Le **predicat** est fourni par une fonction retournant un booléen.

```
l = list(range(10))
for i in itertools.takewhile(lambda x: x < 5, l):
    print(i, end=' ')
0 1 2 3 4

for i in itertools.dropwhile(lambda x: x < 5, l):
    print(i, end=' ')
5 6 7 8 9
```



Modules utiles : **itertools**

- ▶ **itertools** fournit également des méthodes pour obtenir différentes combinaisons des éléments d'une séquence.
- ▶ **itertools.combinations(iterable, r)**
 - ▶ Retourne un itérateur sur les combinaisons non redondantes de **r** éléments de l'**iterable**.

```
colors = ["red", "green", "blue"]
for i in itertools.combinations(colors, 2):
    print(i)
('red', 'green')
('red', 'blue')
('green', 'blue')
```



Modules utiles : **itertools**

- ▶ Si vous avez besoin de manipuler des listes il peut être intéressant de faire un tour dans les fonctions fournies par **itertools**.
- ▶ Celles-ci peuvent souvent simplifier le code en évitant l'utilisation de boucles.



Exercices 5

- ▶ Écrivez un programme qui demande à l'utilisateur d'entrer un nombre (entier ou réel).
 - ▶ Le programme vérifie la validité de l'entrée à l'aide d'une expression régulière.
 - Nombres valides : 8, 888888.5, 0.5, -8
 - Nombres invalides : 8., .5, 8.5.5, - 8
- ▶ Ajoutez le support du *logging* à l'exercice précédent afin que les entrées invalides soient *logguées* dans un fichier avec l'heure à laquelle elles ont été entrées.
- ▶ La facture du restaurant se monte à 100. – et vous avez sur vous 1 billet de 100, 3 de 50, 4 de 20 et 4 de 10.
 - ▶ Quelles sont vos possibilités de régler ce montant ?



Écrire ses propres modules

- ▶ Python permet d'écrire ses propres modules.
- ▶ Il suffit de regrouper les variables, fonctions et classes à l'intérieur d'un fichier dont le nom déterminera l'espace de noms (*namespace*) de ces définitions.
- ▶ Pour l'importer depuis un autre programme, il suffit d'utiliser le nom du fichier sans l'extension.
- ▶ Lors de l'import d'un module Python parcourt les chemins se trouvant dans la variable **PYTHONPATH** afin de trouver le fichier correspondant.



Écrire ses propres modules

- ▶ Lors de l'exécution d'un programme, le chemin vers ce programme se trouvera automatiquement dans la variable **PYTHONPATH**, ainsi si le module à importer se trouve au même niveau tout fonctionnera parfaitement.
- ▶ Sinon, pour que le module soit accessible depuis n'importe quel endroit du système il faut soit :
 - ▶ Placer le module dans l'un des répertoires se trouvant dans le **PYTHONPATH**.
 - ▶ Modifier le **PYTHONPATH** pour qu'il contienne le répertoire où se trouve le module.
 - ▶ Modifier la variable **sys.path** depuis le programme :
 - La modification n'est valide que pour le programme en cours d'exécution.

```
import sys
sys.path.insert(0, path_to_add)
import my_module
```



Écrire ses propres modules

- ▶ Lorsque Python lit un fichier il exécute toutes les instructions qui s'y trouve de la première ligne à la dernière ligne.
- ▶ Avant de démarrer, Python définit certaines variables spéciales.
 - ▶ `__name__` est initialisée avec le nom du module exécuté.
- ▶ Lorsqu'un module est exécuté directement, en tant que programme principal, cette variable est initialisée à la valeur "`__main__`".
- ▶ Ainsi, comme Python ne possède pas de fonction `main()`, le code suivant fait office de "*fonction main() de Python*" :
 - ▶ Dans un module, il permet de tester le module directement.

```
if __name__ == "__main__" :  
    # Main stuff
```



Afficher les symboles importés

- ▶ La fonction `dir()` retourne la liste des symboles définis dans un espace de noms.
- ▶ Sans argument elle retourne les symboles définis dans l'espace de noms courant.
- ▶ Avec lui passant un module (ou un package) en argument elle retourne les symboles définis dans ce module ou package.

```
dir()
['__annotations__', [...], '__spec__']

import os
from time import asctime
dir()
['__annotations__', [...], '__spec__', 'asctime', 'os']

import copy
dir(copy)
[ [...], 'copy', 'deepcopy', [...] ]
```



Limiter l'importation complète

- ▶ L'astérisque ***** ne permet pas d'importer les éléments considérés comme interne au module.
 - ▶ Ceux dont le nom commence par un **_**.
- ▶ Pour définir les éléments qui pourront être importés avec ***** on utilise la variable **__all__** de type **list**.

```
__all__ = ["a", "do_something"]    # Only a and do_something
                                   # will be imported by *

a = 10
b = 20

def do_something():
    print("Something have been done.")

def do_nothing():
    print("zzZZzzzz...")
```



Exercices 6

- ▶ Déplacez les fonctions de l'exercice 4 dans un module et utilisez-les depuis votre programme principal.
 - ▶ Testez l'importation de différentes manières.



Packages

- ▶ Pour créer un package regroupant plusieurs modules il suffit de créer un dossier afin de regrouper les différents fichiers (modules).
- ▶ Ce dossier pourra contenir toute une arborescence de sous-dossiers et de fichiers Python.
 - ▶ Le package n'a pas de limite de profondeur.
- ▶ Un package standard Python contiendra un fichier `__init__.py` dans chaque dossier.
 - ▶ Ce fichier définit le dossier comme étant un package Python.
 - ▶ Dans les cas simples ce fichier est vide.



Packages

- ▶ Un autre type de package ne nécessite pas l'existence du fichier `__init__.py`.
 - ▶ Il s'agit des packages *namespace* (***Namespace packages***).
 - ▶ Une manière particulière de créer des packages qui permet de découper un package en plusieurs packages qui pourront être installés de manière individuelle.
 - ▶ Ceux-ci sont destinés aux personnes qui créent des packages distribuables et ne devraient pas être utilisés dans le cadre de développements personnels.
- ▶ Habituez-vous à mettre des fichiers `__init__.py` dans tous vos packages.



Packages

- Soit l'arborescence de package suivante :

```
lilabyte/  
    __init__.py  
    yop.py  
    functions/  
        __init__.py  
        usefull.py
```

- Avec `yop.py` et `usefull.py` contenant respectivement :

```
a = 10  
b = 20  
  
def do_something():  
    print("Something done.")  
  
def do_nothing():  
    print("zzZZZzzzz...")
```

```
def print_hello():  
    print("Hello World!")  
  
def do_nothing():  
    print("zzZZZzzzz...")
```



Packages

- ▶ Les diverses importations possibles sont par exemple :

```
# Modules importation :  
import lilabyte.yop  
import lilabyte.functions.usefull  
  
# Elements importation :  
from lilabyte.yop import a, do_something  
from lilabyte.functions.usefull import print_hello as hello
```

- ▶ L'importation du package ou du sous-package n'aura aucun effet, tout comme l'importation globale des éléments qu'ils contiennent.
- ▶ En effet, ces importations n'auront eu pour seul effet que d'exécuter les fichiers `__init__.py`.
 - ▶ Et comme ceux-ci sont vides...
- ▶ Mais cela signifie par contre que l'on peut faire quelque chose avec ces fameux fichiers `__init__.py` 😊



__init__.py

- ▶ Le fichier `__init__.py` permet à Python de savoir qu'un dossier contient un module.
- ▶ Il est lu au moment du chargement du module.
 - ▶ Il permet donc d'effectuer certaines initialisations ou définir les symboles qui seront exportés par le module lors d'un import global.
- ▶ Depuis l'intérieur d'un package les modules sont accessibles de manière "absolue" en utilisant le nom du package et le chemin vers le module.
- ▶ Ainsi le code suivant ajouté dans `lilabyte/__init__.py` permettra d'importer le module `yop` lors de l'import du package `lilabyte` :

```
import lilabyte.yop
```



__init__.py

- ▶ Et le code suivant devient valide :

```
import lilabyte  
lilabyte.yop.do_something()
```

- ▶ Pour simplifier encore on pourrait effectuer un import global des éléments de `yop.py` depuis le fichier `__init__.py` :

```
from lilabyte.yop import *
```

- ▶ Et l'appel précédent peut être simplifié :

```
import lilabyte  
lilabyte.do_something()
```



Imports relatifs

- ▶ A l'intérieur d'un package les imports peuvent également être effectués entre des fichiers de modules.
- ▶ Ces imports peuvent être effectués de deux manières :
 - ▶ De manière "absolue" en utilisant le nom du package, des sous-packages et des modules.

```
import lilabyte.yop
import lilabyte.functions.usefull
```

- ▶ Ou de manière relative en démarrant le chemin à importer avec un point (".").
 - Lorsque le chemin commence par deux points ("..") ou plus l'import relatif se fait par rapport au(x) répertoire(s) parent(s), considérant un niveau de parenté par "." après le premier.



Imports relatifs

- ▶ Ainsi pour pouvoir appeler `print_hello()` depuis le module `yop` on effectue un des imports relatifs suivants :

```
from .functions.usefull import print_hello as hello
```

```
from .functions import usefull
```

- ▶ Et pour pouvoir appeler `do_nothing()` depuis le module `usefull` on ferait l'un des imports suivants :

```
from ..yop import do_nothing
```

```
from .. import yop
```



Syntaxe complète

- ▶ Finalement, en appliquant cette dernière approche aux fichiers `__init__.py` et en utilisant la variable `__all__` pour définir ce que l'on veut exporter lors d'un import global on peut créer un package `lilabyte` qui fournit :
 - ▶ Tous les éléments lorsqu'il est importé.
 - ▶ Mais uniquement la variable `a` et les fonctions `do_something` et `print_hello()` lors d'un import global.

```
# lilabyte/functions/__init__.py
from .usefull import *
__all__ = ["print_hello"]
```

```
# lilabyte/__init__.py
from .functions import *
from .yop import *
__all__ = ["print_hello", "a", "do_something"]
```



Package installable

- ▶ La création d'un package Python installable permet de distribuer le résultat d'un développement.
 - ▶ Un programme ou une bibliothèque de classes, fonctions, ...
- ▶ Le processus est réalisé en 3 étapes :
 - ▶ Définition de l'arborescence adéquate.
 - ▶ Ajout des fichiers nécessaires.
 - ▶ Génération du package.



Définition de l'arborescence

- Soit le package défini précédemment placé dans le répertoire **src** de l'arborescence adéquate :

```
package/  
└─ src/  
    └─ lilabyte/  
        ├── __init__.py  
        ├── yop.py  
        └─ functions/  
            ├── __init__.py  
            └─ usefull.py
```



Ajout des fichiers nécessaires

```
package/  
├── LICENSE  
├── pyproject.toml  
├── README.md  
└── src/  
    └── ...
```

► LICENSE

- Le fichier de licence du code
- Pour savoir laquelle choisir :
 - <https://choosealicense.com>
 - Pour l'exemple, la licence **MIT**, comme ce que l'on définira dans le fichier `pyproject.toml`.



Ajout des fichiers nécessaires

- ▶ **README.md**

- ▶ Le fichier représentant la documentation succincte du package au format *Markdown*.

- ▶ Par exemple :

```
# LilaByte Package
```

```
Group all the necessary function for data analyze used  
by LilaByte team.
```

```
See [documentation](https://lilabyte.com/.../doc) for  
more information.
```



Ajout des fichiers nécessaires

- ▶ `pyproject.toml`
- ▶ La conversion des sources en un package installable est réalisée par un backend de génération (*build backend*).
 - ▶ Flit, Hatchling, PDM, Setuptools.
- ▶ Dans cet exemple nous utiliserons Setuptools.
- ▶ Le fichier `pyproject.toml` informe les outils de génération (`pip`, `build`) du backend à utiliser.
- ▶ Il contient :

```
[build-system]
requires = ["setuptools >= 61.0"]
build-backend = "setuptools.build_meta"
```



Ajout des fichiers nécessaires

- Ainsi que le nom du package et d'autres métadonnées :

```
[project]
name = "lilabyte"
version = "0.0.1"
authors = [
    { name = "Philippe Ney", mail = "philippe.ney@lilabyte.com" },
]
description = "LilaByte usefull functions"
readme = "README.md"
requires-python = ">3.10"
classifiers = [
    "Programming Language :: Python :: 3",
    "License :: OSI Approved :: MIT License",
    "Operating System :: OS Independent",
]

[project.urls]
Homepage = "https://lilabyte.com/python/lilabyte"
Issues = "https://lilabyte.com/python/lilabyte/issues"
```



Ajout des fichiers nécessaires

- ▶ La partie **classifiers** fournit à **pip** des informations sur le package.
 - ▶ Ainsi qu'à l'index en cas de déploiement sur PyPI.
- ▶ La partie **[projects.urls]** fournit des liens additionnels qui pourront être affichés sur PyPI.
 - ▶ Elle n'est pas obligatoire.



Génération du package

- ▶ Pour cela il est nécessaire d'installer le package **build** :

```
(venv) jean$ pip install --upgrade build
```

- ▶ Puis depuis le répertoire du fichier **pyproject.toml** :

```
(venv) jean$ python -m build
```

- ▶ Ce qui génère deux fichiers dans le répertoire **dist** :
 - ▶ Un fichier *Source Distribution* :
 - **lilabyte-0.0.1.tar.gz**
 - ▶ Un fichier *Built Distribution* :
 - **lilabyte-0.0.1-py3-none-any.whl**
- ▶ Tous deux peuvent être installés avec **pip**, mais les nouvelles versions préfèrent les fichiers *Built Distribution*.



Installation et distribution

- ▶ Il est maintenant possible d'installer le package en utilisant l'un ou l'autre des fichiers de distribution :

```
(venv) jean$ pip install lilabyte-0.0.1.tar.gz
```

- ▶ Cependant, pour que le package soit accessible à d'autres personnes, la meilleure approche consiste à le déployer sur un serveur de packages.
 - ▶ PyPI ou sur un serveur privé.
- ▶ Le package **devpi** permet de mettre en place un tel serveur compatible avec PyPI permettant également une redirection sur PyPI pour les packages non fournis.



Installation et distribution

- Sinon l'approche la plus simple consiste à utiliser un serveur web classique servant des fichiers statiques avec la structure suivante :

```
.  
└─ package-name/  
   └─ package-file(s)
```

- Par exemple :

```
.  
└─ lilabyte/  
   ├── lilabyte-0.0.1.tar.gz  
   ├── lilabyte-0.0.1-py3-none-any.whl  
   ├── lilabyte-0.0.2.tar.gz  
   └─ lilabyte-0.0.3-py3-none-any.whl  
└─ bytelila/  
   └─ ByteLila-1.0.0-py3-none-any.whl
```



Installation et distribution

- ▶ L'arborescence pourrait être servie en test directement avec le serveur HTTP de Python :

```
(venv) jean$ python -m http.server
```

- ▶ Le package pourra alors être installé avec :

```
(venv) jean$ pip install \  
> --extra-index-url http://localhost:8000 lilabyte
```

- ▶ **--extra-index-url** permet de fournir des URLs additionnelles à celles de PyPI.
- ▶ L'installation s'effectue dans un environnement isolé et **pip** aura besoin d'accéder à un package **setuptools** vérifiant la condition du package à installer.
 - ▶ Sinon l'option **--no-build-isolation** permettra d'utiliser la version de **setuptools** installée dans l'environnement.



Programmation orientée objet

Les bases

Théorie de l'orienté objet

Classe et objet

Attributs et méthodes

Héritage



Intérêt

- ▶ Implique un meilleur design, en général.
- ▶ Plus proche de la réalité :
 - ▶ Les objets ont des caractéristiques :
 - Couleur, taille, etc.
 - ▶ Les objets ont des comportements :
 - Se déplacer, parler, manger, etc.
- ▶ Amélioration significative de la réutilisabilité du code :
 - ▶ Plus facile d'utiliser des modules orientés objets :
 - Distinction claire entre le concepteur du module et son utilisateur.
 - ▶ Plus facile d'étendre un module.



Principes de l'orienté objet

- ▶ En langage procédural le centre du programme est constitué des données.
 - ▶ Et on écrit des fonctions afin de les manipuler.
- ▶ La programmation orientée objet tourne autour d'une seule entité, l'**objet**.
- ▶ Un objet représente une structure de données évoluée :
 - ▶ C'est une entité chargée de gérer des données, soit les classer, les stocker et les manipuler.
- ▶ Un objet se différencie de la notion de structure de données des langages procéduraux dans le sens où :
 - ▶ Il regroupe les données.
 - ▶ Et les moyens de traiter ces données.



Principes de l'orienté objet

- ▶ Chaque objet possède :
 - ▶ Des caractéristiques propres.
 - ▶ Des comportements propres.
- ▶ Les objets sont vus comme des entités indépendantes.
 - ▶ Ils représentent les modules du programme.
- ▶ Le programme est finalement réalisé par des objets qui collaborent entre eux.
 - ▶ Les objets s'envoient des messages.



Le modèle objet

- ▶ La notion d'objet est basée sur l'observation du monde réel qui nous entoure.
- ▶ A chaque objet du monde réel peut correspondre un objet en programmation.
- ▶ Dans le monde réel un objet possède :
 - ▶ Des caractéristiques (ou des états).
 - ▶ Des comportements.
- ▶ En programmation un objet possède :
 - ▶ Des attributs.
 - ▶ Des méthodes.



Le modèle objet

- ▶ Un objet est une entité spécifique.
 - ▶ Ses attributs sont définis.
 - ▶ Ses comportements également.
 - Généralement dû aux attributs.
- ▶ Un objet est d'un certain type.
- ▶ Plusieurs objets différents peuvent être d'un même type de base.
 - ▶ On a une notion de généralisation des objets.
- ▶ Une classe permet de regrouper plusieurs objets d'un même type mais avec des caractéristiques différentes.
 - ▶ On parle de classe d'objets.
 - On utilise le mot réservé **class**.
 - ▶ Une classe représente un modèle permettant de construire des objets.
 - ▶ L'instanciation consiste à créer une instance d'une classe, soit un objet.



Interface et implémentation

- ▶ Les objets interagissent entre eux.
- ▶ Principalement à l'aide de leurs méthodes.
 - ▶ Un objet pourrait cependant également modifier les attributs d'un autre objet.
- ▶ Chaque objet doit donc définir comment il peut être utilisé par les autres objets.
 - ▶ On parle de l'**interface** de l'objet.
- ▶ La manière dont l'objet exécute une méthode (ou répond à un message) est propre à l'objet.
 - ▶ On parle de l'**implémentation** de l'objet.
- ▶ L'interface est la partie visible de l'objet et l'implémentation la partie cachée.



Interface et implémentation

- ▶ Un des buts de l'orienté objet et de bien séparer l'interface de l'implémentation afin de présenter l'objet comme une boîte noire à ceux qui l'utilise.
- ▶ Cette séparation est nécessaire :
 - ▶ Elle permet de cacher l'implémentation.
 - ▶ Et protège ainsi l'objet contre des accès non désirés.
 - ▶ Elle permet également de modifier un comportement, voir l'optimiser sans agir sur le reste du système :
 - L'implémentation doit respecter la définition de l'interface et ne pas la modifier.
 - Il est donc possible de développer une structure et effectuer des optimisations en fin de développement, ce qui correspond bien aux modèles de développement agile d'aujourd'hui.
- ▶ Cette séparation est réalisée par la notion d'encapsulation qui définit quels sont les éléments accessibles ou inaccessibles d'un objet.



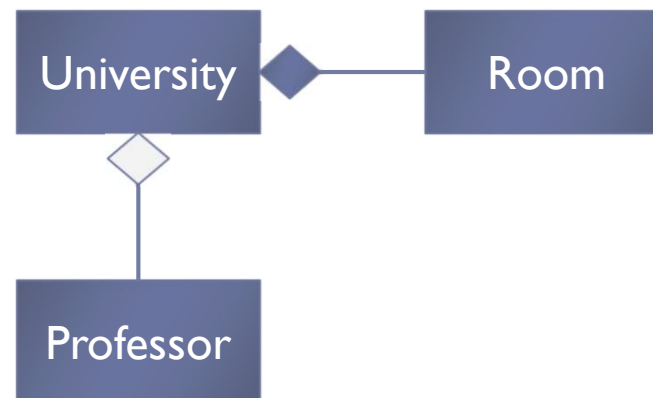
Réutilisation

- ▶ Une fois une classe définie, l'intérêt est de pouvoir la réutiliser.
- ▶ C'est un des grands avantages de l'OO.
- ▶ Une classe étant caractérisée par :
 - ▶ Son interface.
 - ▶ Son implémentation.
- ▶ Les deux aspects pourront être réutilisés
 - ▶ Réutilisation de l'interface :
 - On parle de l'**héritage**.
 - ▶ Réutilisation de l'implémentation :
 - On parle de l'**agrégation** ou de la **composition**.



Réutilisation

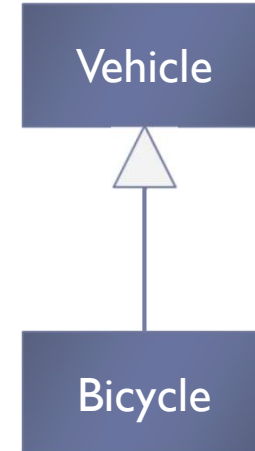
- ▶ L'agrégation et la composition représentent une relation "**constitué-constituant**" entre deux classes.
- ▶ Elle est représentée par les phrases "**possède un**" ou "**fais partie de**" :
 - ▶ Un vélo possède des roues.
 - ▶ Une chambre fait partie d'un appartement.
- ▶ La composition représente une agrégation forte où le constituant ne peut pas exister sans le constitué.





Réutilisation

- ▶ La relation d'héritage représente un **spécialisation** ou une **généralisation** entre deux classes.
- ▶ On parle de la classe parent et de la classe fille.
- ▶ La classe fille hérite des propriétés accessibles de la classe parente.
 - ▶ Si nécessaire, elle pourra les écraser (modifier).
- ▶ Elle pourra en plus ajouter ses propres caractéristiques et comportements.
- ▶ Elle est représentée par la phrase "**est un**" :
 - ▶ Un vélo est un véhicule





Objets et classes

- ▶ Un objet représente une entité particulière, unique.
 - ▶ Ma voiture, John, la maison de Paul, etc..
- ▶ Une classe représente un ensemble d'objets ayant des caractéristiques et des comportements communs.
 - ▶ Ma voiture a des attributs communs avec toutes les voitures (elles ont une couleur, un nombre de sièges, de portes, etc.).
 - ▶ Les voitures partagent également certains comportements (elles peuvent rouler, freiner, tourner, etc.).
- ▶ Par convention :
 - ▶ Le nom des classes s'écrit en **PascalCase** (**Car**) et celui des objets en **lower_case_with_underscores** (**my_car**).



Constructeur

- ▶ Une classe est un modèle qui permet de créer des objets.
 - ▶ C'est un nouveau type de données.
 - ▶ On utilise le mot réservé **class** pour définir une classe.
 - ▶ Lors de la création d'un objet à partir d'une définition de classe on parle d'instanciation.
- ▶ Lors de la création d'un objet d'une classe on utilise le nom de la classe avec des parenthèses.
 - ▶ Cet appel est transféré à une méthode spéciale de la classe, la méthode **`__init__()`** qu'on appelle **le constructeur** de la classe.
 - ▶ Une classe possède un constructeur par défaut, il n'est donc pas nécessaire de définir cette méthode.

```
class Point:
    pass

p = Point()           # Create a new Point
```



Attributs et méthodes

- ▶ Un objet possède des caractéristiques et des comportements.
- ▶ Ceux-ci sont modélisés par les membres de la classe :
 - ▶ Les attributs de la classe sont les caractéristiques de l'objet.
 - ▶ Les méthodes de la classe correspondent à ses comportements.
- ▶ Pour ajouter une méthode à une classe il suffit de définir une fonction (**def**) à l'intérieur de la classe.
- ▶ Pour ajouter un attribut d'instance (ou variable d'instance) à une classe, soit un attribut qui pourra être différent pour chaque objet de la classe, il faudra posséder une référence vers l'objet créé.
 - ▶ Cette définition ne pourra être effectuée qu'à l'intérieur d'une méthode de la classe.
 - ▶ Généralement dans le constructeur.



Attributs et méthodes : **self**

- ▶ Pour ajouter une méthode à une classe il suffit de définir une fonction (**def**) à l'intérieur de la classe.
- ▶ Avec la condition que cette méthode possède toujours au moins un paramètre qui fera référence à l'objet courant.
 - ▶ Ce paramètre doit être présent pour toutes les méthodes.
 - ▶ Il doit être le premier paramètre.
 - ▶ Son nom n'est pas défini mais par convention on utilise **self**.

```
class Point:  
    def initialize(self):  
        pass
```

- ▶ Ce paramètre permettra également d'appeler d'autres méthodes depuis l'intérieur de la classe.
 - ▶ Par contre, lors de l'appelle de la méthode ce paramètre est **ignoré**.



Attributs et méthodes : **self**

- ▶ Pour créer une variable d'instance il est nécessaire d'utiliser une référence sur l'objet courant.
- ▶ Cette référence n'étant accessible que dans les méthodes de la classe, une variable d'instance ne pourra être créée que dans une méthode de la classe.
- ▶ La définition d'une variable d'instance sera donc effectuée de la manière suivante :

```
class Point:  
    def initialize(self, x, y):  
        self.x = x  
        self.y = y
```

- ▶ Le premier paramètre des méthodes (**self**) est donc utilisé à l'intérieur de la classe pour accéder tant les variables que les méthodes.



Constructeur

- ▶ Lors de la création d'un objet Python appellera automatiquement le constructeur par défaut de la classe.
 - ▶ Dont le nom est `__init__(self)`.
- ▶ Si l'on désire effectuer des initialisations au moment de la création d'un objet, il suffit de redéfinir le constructeur :
 - ▶ C'est également dans le constructeur que sont effectuées la création et l'initialisation des variables d'instance.

```
class Point:
    def __init__(self, x=0, y=0):
        self.x = x
        self.y = y

p1 = Point()           # Create a new Point at (0, 0)
p2 = Point(10, 20)     # Create a new Point at (10, 20)
```



Destructeur

- ▶ Avec Python, la gestion de la mémoire est effectuée automatiquement à l'aide d'un *garbage collector*.
- ▶ Il est donc moins important qu'en C++ d'avoir la notion de destructeur.
- ▶ Cependant comme la mémoire allouée n'est pas le seul paramètre dont il faut se soucier, la notion de destructeur existe également.
 - ▶ Il est parfois nécessaire de fermer des connexions réseau (socket) ou des connexions à des bases de données, etc.
- ▶ Le destructeur ne prend bien évidemment aucun paramètre :

```
class Point:
    def __init__(self, x=0, y=0):           # Constructor
        print("Created")
    def __del__(self):                       # Destructor
        print("Destroyed")
```



Destructeur

- ▶ Le *garbage collector* de Python est de type *reference-counting*, ce qui signifie qu'un objet est libéré de la mémoire lorsqu'il n'y a plus aucune référence sur lui.
 - ▶ Ce qui se produit lorsque les variables sont détruites parce qu'elle sortent de leur portée.
 - ▶ Ou de toute manière à la fin du programme.
- ▶ Pour libérer un objet plus rapidement, le plus simple est de lui assigner la valeur **None**.



Variable de classe

- ▶ Une **variable de classe** (ou attribut de classe) est une variable **identique** pour tous les objets de la classe.
 - ▶ Il ne s'agit pas d'une variable présente dans tous les objets et qui aurait la même valeur, mais bien d'une seule variable en mémoire à laquelle tous les objets d'une même classe pourront accéder.
- ▶ Pour définir une variable de classe il suffit de la définir à l'extérieur de toute méthode :

```
class Point:  
    x = 0  
    y = 0
```

- ▶ Une telle variable ne pourra ensuite être modifiée qu'en utilisant un accès par classe (**ClassName.attributName**) soit :

```
Point.x = 10  
Point.y = 20
```

- ▶ Et ceci **quelque soit l'endroit** depuis lequel la variable est modifiée. Que ce soit depuis l'intérieur de la classe ou depuis l'extérieur.



Exercices 7

- ▶ Créez une classe permettant de représenter un animal familier (`class Pet`) qui a les caractéristiques et les comportements suivants :
 - ▶ Il a un âge, un poids, un nom.
 - ▶ Il peut manger une quantité de "pâtée pour pets", ce qui le fera grossir de 20% de la quantité ingurgitée.
 - ▶ Il peut également dormir et parler.
 - ▶ Lorsqu'on le fait parler, il est possible de lui passer en paramètre ce qu'il doit dire.
 - ▶ Ajoutez un compteur automatique de manière à savoir en tout temps le nombre de `Pet` créés.



Réutilisation du code

- ▶ Un des grands avantages de la programmation orientée objet consiste en la réutilisation du code.
- ▶ Une fois qu'une classe a été définie, il est possible de l'utiliser de trois manières :
 - ▶ En créant des objets de cette classe dans le programme.
Utilisation classique.
 - ▶ En créant des objets de cette classe à l'intérieur d'une autre classe.
On parle alors d'agrégation ou de composition.
 - Une voiture (`class Car`) possède un moteur (`class Motor`) ainsi que des portes (`class Door`) et des sièges (`class Seat`), ...
 - Un bâtiment Universitaire (`class Building`) est constitué de classe (`class Room`) et des professeurs (`class Professor`) y travaillent.
 - ▶ En définissant une nouvelle classe qui héritera des propriétés de cette classe, qui sera considérée comme sa classe parente.
On parle alors d'héritage.
 - Les professeurs sont des personnes (`class Person`), ils héritent donc des propriétés (caractéristiques et comportements) de cette classe.



Héritage

- ▶ L'héritage permet de récupérer et d'**étendre** et/ou **modifier** les fonctionnalités d'une classe.
- ▶ En partant d'une classe **parente** générique, l'héritage permettra de définir une ou plusieurs classes **filles** spécifiques.
- ▶ L'héritage permet de créer une hiérarchie afin de décrire le problème en profitant au maximum de la notion d'abstraction.
- ▶ La syntaxe de l'héritage est la suivante :

```
class Form:                # Base class
    pass

class Square(Form):        # Square inherit from Form
    pass
```



Appel des constructeurs

- ▶ Lors de l'instanciation d'un objet d'une classe dérivée, seul le constructeur par défaut de la classe parente sera appelé automatiquement.
 - ▶ Si celui-ci a été réécrit (le constructeurs par défaut de la classe parent), il ne sera pas appelé automatiquement.
- ▶ Cependant, si la classe fille ne redéfinit pas de constructeur, elle **hérite du constructeur de la classe parente**.
- ▶ Ainsi, si l'instanciation de la classe parent nécessite l'utilisation d'un autre constructeur, il sera nécessaire d'effectuer un appel au constructeur de la classe parent.
- ▶ Cet appel est effectué à l'aide de la fonction **super ()** qui permet l'accès à la classe parente.



Appel des constructeurs

- On a donc, dans le cas de la hiérarchie suivante :

```
class BaseClass:
    def __init__(self, p1, p2):
        pass

class DerivedClass(BaseClass):
    def __init__(self, p1, p2, p3):
        super().__init__(p1, p2)
        # Do something clever with p3
```

- Mais si la classe fille ne prend pas plus de paramètres que la classe parente, la définition du constructeur n'est pas nécessaire et le constructeur de la classe parente sera appelé automatiquement.



Héritage multiple

- ▶ Python, à l'inverse de Java, autorise l'héritage multiple.
 - ▶ Une classe peut donc hériter des propriétés de plusieurs autres classes.
- ▶ Et Python le fait bien.
 - ▶ En définissant un ordre de résolution des appels (**MRO** pour *Method Resolution Order*) Python permet de résoudre le fameux problème du "Diamond of death".
- ▶ Le MRO est visible dans l'attribut `__mro__` de la classe :

```
class A:
    pass
class B:
    pass
class X(A, B):
    pass

print(X.__mro__)
```



Héritage multiple

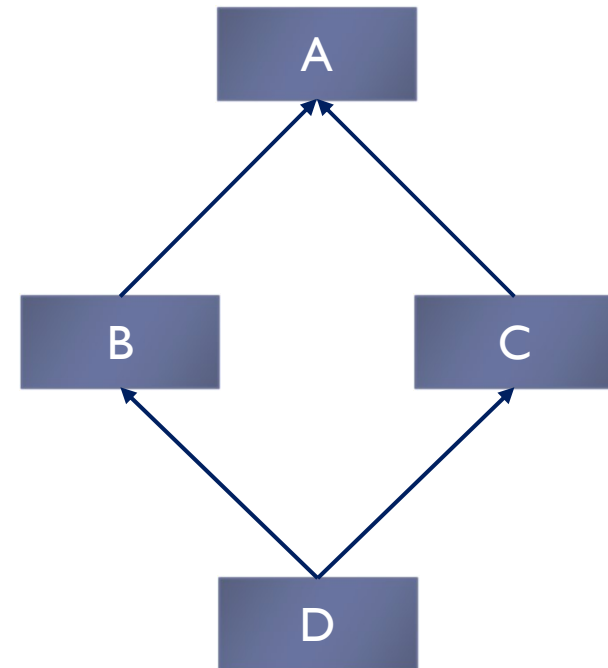
- La logique de définition du *MRO* est basée sur l'ordre d'héritage utilisé lors de l'écriture du code.

```
class A:
    def do_smth(self):
        print("do_smth from A")

class B(A):
    def do_smth(self):
        print("do_smth from B")

class C(A):
    def do_smth(self):
        print("do_smth from C")

class D(B, C):
    pass
```





Héritage multiple

- ▶ Lors de l'appel de la méthode `do_smth` sur un objet de la classe `D`, on peut se demander quelle méthode sera appelée.
 - ▶ Celle de la classe `B`, celle de la classe `C` ou celle de la classe `A` ?
- ▶ La réponse est donnée par le *MRO* :

```
d = D()
d.do_smth()
do_smth from B

print(D.__mro__)
(<class '__main__.D'>, <class '__main__.B'>, <class
 '__main__.C'>, <class '__main__.A'>, <class 'object'>)
```



Exercices 8

- ▶ Créez des classes dérivées pour définir des animaux particuliers (**Dog**, **Cat**, **Fish**).
 - ▶ L'intérêt étant de pouvoir écraser des fonctions de la classe de base dans les classes dérivées.
 - ▶ Vous devriez maintenant être en mesure de faire aboyer le chien et miauler le chat.
- ▶ Sauvegardez vos animaux dans un tableau et faites-les parler à l'aide d'une boucle.



Gestion des exceptions

Attraper les exceptions

Lancer des exceptions

Définir ses propres exceptions



Introduction

- ▶ Lorsqu'une erreur est détectée en cours d'exécution d'un programme, l'interpréteur génère une exception.
 - ▶ L'exécution est interrompue et un message d'erreur s'affiche :

```
>>> print(10 / 0)
ZeroDivisionError: division by zero
>>> val = float("Hello")
ValueError: could not convert string to float: 'Hello'
```

- ▶ Le message d'erreur donne 2 informations :
 - ▶ Le type de l'erreur (**ZeroDivisionError**, **ValueError**).
 - ▶ Un message explicatif.
- ▶ L'intérêt des exceptions est qu'il est possible de les attraper et de les traiter afin d'éviter que le programme ne se termine.



Attraper les exceptions

- ▶ Lorsque des instructions sont susceptibles de générer des exceptions, on exécute celles-ci à l'intérieur d'un gestionnaire d'exceptions qui permettra de traiter l'exception générée.
- ▶ Lorsqu'une exception est générée le flux normal d'exécution des instructions est stoppé et le contrôle est passé au gestionnaire d'exceptions.
- ▶ Si le gestionnaire ne traite pas l'exception générée, celle-ci se propage à la couche supérieur du programme ou elle pourra être traitée par un autre gestionnaire d'exceptions.
 - ▶ Si elle arrive dans le flux principal sans être traitée, elle générera l'arrêt du programme.



Attraper les exceptions

- ▶ Un gestionnaire d'exceptions est constitué des instructions **try-except-else-finally**.
- ▶ Les instructions qui sont susceptibles de générer des exceptions sont exécutées dans le bloc **try**.
- ▶ Si une exception se produit, le flux de contrôle est passé au bloc **except** dédié à la gestion de cette exception.
 - ▶ Si aucun bloc **except** ne gère ce type d'exception, le flux est passé au programme appelant.
- ▶ Si aucune exception ne se produit alors les instructions du bloc **else** sont exécutées.
- ▶ Les instructions du bloc **finally** sont toujours exécutées.
- ▶ Les blocs **else** et **finally** sont optionnels.
- ▶ Il peut y avoir plusieurs blocs **except** en fonction des différents types d'exceptions que l'on désire traiter.



Attraper les exceptions

- ▶ Récupération de toutes les exceptions :

```
d = {'first': "Julien", 'name': "Ambord"}
try:
    print(d['birthdate'])
except:
    print("Key not found")
```

- ▶ Récupération de différentes exceptions :
 - ▶ Avec récupération de l'objet exception généré.

```
try:
    open("path_to_file")
    print(d['birthdate'])
except KeyError as e:
    print("KeyError")
    print(e)
except FileNotFoundError as e:
    print(e)
```



Attraper les exceptions

- ▶ Lors de la récupération de plusieurs types d'exceptions, les gestionnaires doivent être placés du plus spécifique au plus général.
- ▶ Cela est dû à la hiérarchie des classes d'exception.

```
try:
    print(d['birthdate'])
except KeyError:                # KeyError is a LookupError
    print("KeyError")
except LookupError:            # LookupError is an Exception
    print("LookupError")
except Exception:
    print("Error")
```



Exercices 9

- ▶ Modifiez votre code de l'exercice 5 afin d'utiliser une approche plus *pythonique*, basée sur les exceptions.
 - ▶ Utilisez une approche **EAFP** plutôt que **LBYL**.
 - ▶ Les entrées `.8` et `8.` seront cette fois considérées valides.



Lancer une exception

- ▶ Pour lancer une exception on utilise l'instruction **raise** avec pour argument soit un objet de type **Exception**, soit un classe dérivant de **Exception**.
- ▶ En utilisant une classe dérivant de **Exception**, celle-ci sera automatiquement instanciée en appelant son constructeur.
- ▶ il est donc possible de lui passer un message d'information :

```
raise ValueError("Value outside bounds")  
raise ValueError                                # Short call
```



Transférer une exception

- ▶ Il est parfois nécessaire de transférer une exception au niveau supérieur du programme.
- ▶ Le gestionnaire doit capturer l'exception afin d'effectuer des manipulations particulières puis transmettre cette exception au code appelant.
- ▶ Pour cela l'exception est relancée avec un simple **raise** :

```
try:
    raise ValueError("Value outside bounds")
except ValueError:
    print("Exception caught: Do something clever")
    raise
```



Définir ses propres exceptions

- ▶ Avec Python il est très simple de définir ses propres exceptions.
- ▶ Il suffit de créer une classe qui hérite de **Exception** :

```
class MyException(Exception):  
    pass
```

- ▶ Qu'il suffira ensuite de lancer avec **raise** en lui fournissant le message informatif :

```
raise MyException("Something went wrong!")
```



Définir ses propres exceptions

- ▶ Si l'on désire ajouter plus d'information à l'exception, il suffit de réécrire le constructeur en ajoutant des paramètres :

```
class MyException(Exception):  
    def __init__(self, message, errors=None):  
        super().__init__(message)  
        self.errors = errors
```

- ▶ Afin de lui passer des données qui pourront être récupérées lorsque l'exception se produit :

```
except MyException as e:  
    print(e.errors)
```



Chaînage d'exceptions

- ▶ La *stack trace* montre les exceptions produites lors d'une suite d'exceptions (exception lancée depuis un bloc **except**) :

```
__main__.MyException: Problem...  
During handling of the above exception, another exception occurred:  
SyntaxError: Error
```

- ▶ Il est possible d'avoir un message plus parlant en utilisant l'instruction **from** lors du lancement d'une exception :

```
except MyException as e:  
    raise MyException("Error") from e
```

- ▶ Qui produira :

```
__main__.MyException: Problem...  
The above exception was the direct cause of the following exception:  
SyntaxError: Error
```



Nettoyages prédéfinis

- ▶ La clause **finally** est utilisée pour exécuter des instructions qui doivent être effectuées quel que soit le résultat de l'exécution des instructions précédentes.
 - ▶ Typiquement le fait de fermer un fichier ouvert.
- ▶ Dans ce cas on utilise l'instruction **with** qui s'occupera de fermer le fichier quel que soit le résultat des instructions précédentes.
- ▶ La fermeture du fichier est exécutée automatiquement dans une clause **finally** encapsulée qui, en cas d'exception, s'occupera de la propager.
- ▶ Ainsi si un problème se produit lors de la manipulation d'un fichier, il faudra également penser à gérer les exceptions qui pourraient être produites.



Clause **else** des boucles et du gestionnaire d'exceptions



Clause **else** dans les boucles

- ▶ Les boucles **for** et **while** ainsi que le gestionnaire d'exceptions possèdent une clause **else**.
 - ▶ `try: -> except: -> else: -> finally:`
- ▶ Elle s'exécute lorsque la boucle se termine de manière normale ou qu'aucune exception ne s'est produite.
 - ▶ Pour la boucle **for** lorsque la séquence a été parcourue avec succès.
 - ▶ Pour la boucle **while** lorsque la condition devient **False**.
 - ▶ Soit lorsque la boucle n'a pas été terminée par un appel à **break**.
- ▶ La clause **else** peut être intéressante suivant la logique à mettre en place.
 - ▶ Attention cependant à bien comprendre son fonctionnement.



Décorateurs



Présentation

- ▶ Les décorateurs sont des fonctions qui permettent de modifier le comportement par défaut d'autres fonctions, voir même de classes.
- ▶ Un décorateur prendra une fonction en paramètre et retournera une fonction.
 - ▶ La fonction retournée peut être la même que celle reçue ou être une autre fonction générée par le décorateur.
- ▶ Ainsi lors de l'appel d'une fonction décorée, celle-ci ne s'exécutera pas elle-même, mais exécutera le décorateur qui décidera si il faut exécuter la fonction et dans quelles conditions ou si il faut exécuter une autre fonction ou ne rien faire.



Syntaxe

- Pour décorer une fonction, on ajoute **au-dessus** de sa définition le signe @ suivi du nom du décorateur :

```
def my_decorator(function) :  
    return function  
  
@my_decorator  
def say_hello() :  
    print("Hello world")
```

- Il est possible d'ajouter plusieurs décorateurs à une fonction.
 - L'ordre est important car il définit l'ordre d'exécution des fonctions décoratrices.

```
@my_decorator_1  
@my_decorator_2  
@my_decorator_3  
def say_hello() :  
    print("Hello world")
```



Syntaxe

- ▶ La syntaxe de définition d'un décorateur à l'aide du signe @ est une syntaxe simplifiée de :

```
def my_decorator(function):  
    return function  
  
def say_hello():  
    print("Hello world")  
  
say_hello = my_decorator(say_hello)
```

- ▶ Ainsi un décorateur s'exécute au moment de la **définition** de la fonction et **non** au moment de son exécution.
 - ▶ Cette subtilité est très importante car elle définit la manière d'écrire le code d'un décorateur.
 - ▶ Pour **modifier** une fonction le décorateur doit donc définir une **nouvelle** fonction qui sera retournée.



Syntaxe

- ▶ Exemple de décorateur qui affiche le nom et l'adresse d'une fonction avant que celle-ci s'exécute :

```
def my_decorator(function):  
  
    def inner():  
        print(f"Calling {function}")  
        function()  
  
    return inner  
  
@my_decorator  
def say_hello():  
    print("Hello world")
```

- ▶ En affichant la fonction `say_hello`, la décoration sera affichée :

```
<function __main__.my_decorator.<locals>.inner()>
```



Exercices 10

- ▶ Écrivez un décorateur qui empêche l'exécution d'une fonction lorsque les secondes sont impaires.
- ▶ L'exécution de la fonction affichera l'heure courante et :
 - Dans le cas de secondes paires, l'output de la fonction.
 - Dans le cas de secondes impaires, un message explicatif.



Décorateur paramétré

- ▶ Si l'on désire pouvoir passer des paramètres au décorateur la syntaxe devient :

```
@my_decorator(param)
def say_hello():
    print("Hello world")
```

- ▶ Ce qui correspond à :

```
say_hello = my_decorator(param)(say_hello)
```

- ▶ Et visiblement le décorateur **ne peut plus** prendre la fonction en paramètre... Flûte !
- ▶ La solution consiste à augmenter le niveau d'imbrication :
 - ▶ Le décorateur renverra un autre décorateur qui pourra, lui, prendre la fonction en paramètre.



Décorateur paramétré

► Exemple de décorateur paramétré :

```
def my_decorator(nb):  
  
    def decorator(function):  
  
        def inner():  
            print(f"Calling {nb} times {function}")  
            for i in range(nb):  
                function()  
  
        return inner  
  
    return decorator  
  
@my_decorator(4)  
def say_hello():  
    print("Hello world")
```

► Quel sera l'affichage produit par ce programme ?



Exercices 11

- ▶ Écrivez un décorateur qui délimite la sortie produite par une fonction avec des lignes constituées d'un nombre variable de caractères spéciaux.
- ▶ Les paramètres du décorateur sont le nombre et le caractère à utiliser pour créer les lignes.
- ▶ La sortie pourrait être du style :

```
*****  
Hello world !  
*****
```



Décorateur de fonctions paramétrées

- ▶ Pour l'instant nous avons décoré des fonctions simples, qui ne prennent pas de paramètres.
- ▶ Comment décorer des fonctions paramétrées ?
- ▶ Si l'on connaît le nombre de paramètres, le problème est simple, il suffit de créer la fonction interne avec le même nombre de paramètres.
 - ▶ De cette manière elle pourra les passer à la fonction à décorer lors de son exécution (si nécessaire).



Décorateur de fonctions paramétrées

- ▶ Mais de cette manière on perd une caractéristique très importante du décorateur, **sa généricité**.
- ▶ Heureusement, Python fournit une notation spécifique permettant de récupérer les paramètres de la fonction dans le décorateur :

```
def inner_function(*args, **kwargs) :
```

- ▶ **args** est un tuple des arguments positionnels et
- ▶ **kwargs** un dictionnaire des arguments nommés.
- ▶ Il suffit de transmettre ces arguments à la fonction à exécuter.



Correction des métadonnées

- ▶ Le problème avec les fonctions décorées est qu'elles perdent souvent les métadonnées qui leurs sont associées.
- ▶ Soit le programme suivant :

```
def my_decorator(function):  
    def inner():  
        function()  
    return inner  
  
@my_decorator  
def say_hello():  
    """Function that says hello to the world!"""  
    print("Hello world")  
  
print(say_hello)  
print(say_hello.__name__)  
print(say_hello.__doc__)
```



Correction des métadonnées

- ▶ La sortie produite par le programme sera la suivante :

```
<function my_decorator.<locals>.inner at 0x7f57e0863af0>  
inner  
None
```

- ▶ Les information relatives à la fonction **say_hello()** ont donc été perdues durant la décoration.
- ▶ Pour éviter cela on utilise un décorateur du module **functools** afin de décorer **la fonction retournée**.
 - ▶ De cette manière les métadonnées de la fonction originelle seront transmises à la fonction retournée par le décorateur.
 - La modification et le résultat sont donnés à la slide suivante.



Correction des métadonnées

- ▶ Avec l'utilisation du décorateur `functools.wraps()` :

```
import functools

def my_decorator(function):
    @functools.wraps(function)
    def inner():
        [...]

print(say_hello)
print(say_hello.__name__)
print(say_hello.__doc__)
```

```
<function say_hello at 0x7f7e5154faf0>
say_hello
Function that says hello to the world!
```

- ▶ Toutes les informations sont conservées !



Exercices 12

- ▶ Modifiez le décorateur de l'exercice précédent afin de pouvoir utiliser une fonction prenant en paramètre le texte à afficher.
 - ▶ Dans le décorateur, affichez les paramètres reçus pour la fonction décorée.
- ▶ Ecrivez un décorateur qui permette de tracer le temps d'exécution de n'importe quelle fonction.
 - ▶ L'output de la fonction doit évidemment pouvoir être récupéré lors de son exécution.
 - ▶ Testez avec une fonction permettant de suspendre l'exécution du programme, telle que `input()` ou `time.sleep()`.



Programmation orientée objet Suite

Encapsulation et propriétés

Membres statiques

Méthodes abstraites

Réécriture d'opérateurs



Méthodes spéciales

- ▶ Une classe contient deux méthodes spéciales intéressantes :

<code>__repr__()</code>	Représentation "officielle" de l'objet en tant que chaîne. Automatiquement appelée par la fonction <code>repr()</code> .
<code>__str__()</code>	Représentation de l'objet en tant que chaîne. Automatiquement appelée par la fonction <code>str()</code> .

- ▶ Lorsque `__str__` n'existe pas, `__repr__` est appelée automatiquement.
- ▶ Le but de `__repr__` est de représenter l'objet de manière univoque.
 - ▶ Elle devrait fournir une expression Python pouvant être utilisée pour créer un clone de l'objet.
 - ▶ Elle est par exemple appelée lors de l'affichage d'un objet dans le *shell* Python.
 - ▶ Normalement pour un objet `o`, le test "`o == eval(repr(o))`" devrait retourner `True`.
- ▶ `__str__` sera appelée lors de l'affichage d'un objet par `print()`.



Encapsulation : Public, protected, private

- ▶ Une des caractéristiques de la programmation orientée objet est la notion d'encapsulation qui offre plein d'avantages (limite les bugs, optimisation, etc.).
- ▶ Avec Python il n'est pas possible d'encapsuler **réellement** les attributs et les méthodes d'une classe.
 - ▶ Ces membres seront toujours accessibles.
- ▶ Il existe cependant certaines approches permettant de simuler l'encapsulation.
- ▶ Pour ce faire on utilise la notation suivante (exemple avec un attribut **name**) :

name	Attribut public . Accessible depuis l'intérieur et l'extérieur de la classe.
_name	Attribut protected . Ne devrait pas être accédé depuis l'extérieur de la classe, sauf depuis les classe dérivées.
__name	Attribut private . Inaccessible et invisible depuis l'extérieur de la classe.



Encapsulation : Public, protected, private

- ▶ En ajoutant un trait souligné (name) au début du nom d'un membre de la classe (attribut ou méthode) on signifie que le membre ne devrait pas être utilisé.
 - ▶ Le membre est cependant toujours accessible depuis partout.
- ▶ En ajoutant un double trait souligné (name) Python va automatiquement renommer le membre en y ajoutant le nom de la classe : ClassNamemember
- ▶ Ce renommage a pour but de garantir que les classes dérivées n'écrasent pas accidentellement les méthodes et attributs privés de leur classe parent. Il n'est pas conçu pour empêcher un accès délibéré de l'extérieur.
 - ▶ Le membre est toujours accessible, mais en utilisant son nom brouillé.



Encapsulation : Getter, setter et propriété

- ▶ L'intérêt de rendre privés les attributs d'une classe est d'éviter qu'ils ne soient accédés d'une mauvaise manière.
 - ▶ Typiquement au moment de l'assignation.
- ▶ Dans ce cas la classe doit fournir des méthodes permettant de les accéder.
 - ▶ C'est ce qu'on appelle les **getter** et les **setter**.
- ▶ Par exemple, pour une classe possédant l'attribut privé **__age** :
 - ▶ Le **setter** permettra de vérifier la validité de l'âge.

```
class A:
    def __init__(self, age):
        self.__age = age

    def get_age(self):
        return self.__age

    def set_age(self, age):
        # Do stuff to check that age is valid
        self.__age = age
```



Encapsulation : Getter, setter et propriété

- ▶ Cette approche est la bonne en terme de programmation OO.
- ▶ Cependant si cette écriture provient de la refactorisation d'une classe qui était utilisée précédemment par d'autres personnes, d'autres parties de code, ou autre, cela pourrait poser des problèmes.
 - ▶ En effet, l'approche Python étant de ne rien fermer, il est possible qu'un programme n'ait pas utilisé la notion d'encapsulation.
- ▶ C'est pourquoi Python offre une meilleure manière de gérer les accès aux attributs privés.
 - ▶ Celle-ci permet de plus un accès bien plus intuitif.
- ▶ Cette manière s'appelle les **propriétés** et pour les définir on utilise des décorateurs de méthodes.



Encapsulation : Getter, setter et propriété

- ▶ Un décorateur est défini sur une fonction ou une méthode et permet de modifier son comportement :
 - ▶ Il est ajouté juste au-dessus de la déclaration de la fonction.
 - ▶ Il commence par une arobase (@).

```
class A:  
    @decorator  
    def do_something(args):
```

- ▶ Pour définir les propriétés associées à un attribut on utilisera deux décorateurs :
 - ▶ `@property` Permet de définir le **getter**.
 - ▶ `@attribut.setter` Permet de définir le **setter**.



Encapsulation : Getter, setter et propriété

- On a donc finalement le code suivant :

```
class A:
    def __init__(self, age):
        self.__age = age

    @property
    def age(self):
        return self.__age

    @age.setter
    def age(self, age):
        if age < 0:
            self.__age = 0
        elif age > 150:
            self.__age = 150
        else:
            self.__age = age
```

- Qui permet d'accéder l'attribut **age** de manière naturelle, comme on pourrait s'y attendre, mais tout en effectuant des contrôles de validité sur les valeurs passées.



Propriétés : Effacement

- ▶ Comme on l'a déjà vu, il est tout à fait possible d'ajouter des attributs à un objet de manière dynamique.
- ▶ Mais il sera également possible d'en effacer :

```
a = A(22)
a.name = "Julien"      # Add a name attribute to object a
del(a.age)             # Delete the age attribute from object a
```

- ▶ En utilisant les propriétés, l'effacement ne sera plus possible.
- ▶ Et l'on pourra même faire mieux, avec le décorateur `@deleter`, il sera possible de définir comment agir lorsque l'attribut est effacé :

```
class A:
    ...
    @age.deleter
    def age(self):
        self.__age = 0
```



Membres statiques

- ▶ Comme on l'a vu précédemment, pour créer des attributs statiques il suffit de les définir dans la classe à l'extérieur de toute méthode.
- ▶ Il devient alors intéressant de pouvoir créer des méthodes statiques ou des méthodes de classe.
 - ▶ Pour accéder aux attributs statiques, ce qui est plus propre.
 - ▶ Pour les utiliser sans devoir créer d'instance de la classe.
- ▶ Une méthode statique pouvant être appelée sans objet, elle ne peut pas prendre de paramètre **self** étant donné que celui-ci peut ne pas exister.
- ▶ Mais pour l'instant il n'est pas possible de créer une méthode sans ce paramètre.



Membres statiques

- ▶ La solution est donnée par l'utilisation de décorateurs.
- ▶ Un décorateur est défini sur une fonction ou une méthode et permet de modifier son comportement :
 - ▶ Il est ajouté juste au-dessus de la déclaration de la fonction.
 - ▶ Il commence par une arobase (@).

```
class A:  
    @decorator  
    def do_smth(args) :
```

- ▶ Pour définir une méthode de classe on utilise le décorateur **@classmethod**.
- ▶ Pour définir une fonction statique on utilise le décorateur **@staticmethod**.



Méthodes statiques et méthode de classe

► Mais quelle est la différence ?

- Une méthode statique n'est liée à rien, ni à un objet, ni à une classe.
- Une méthode de classe n'est pas liée à un objet, mais à une classe.

```
class A:
    def foo(self, x):
        print(f"{self}, {x}")

    @classmethod
    def class_foo(cls, x):
        print(f"{cls}, {x}")

    @staticmethod
    def static_foo(x):
        print(f"{x}")
```

- Ainsi, lorsqu'une méthode doit accéder à d'autres éléments de la classe, on préférera définir une méthode de classe, ce qui évitera de devoir écrire le nom de la classe en dur dans le code, mais permettra l'utilisation du paramètre **cls** de la méthode.



Exercices 13

- ▶ Faites le nécessaire afin de pouvoir afficher facilement les informations d'un **Pet**.
- ▶ Ajoutez à votre programme une méthode permettant de récupérer le nombre de **Pet** créés.
- ▶ Encapsulez les informations importantes (nom, poids, âge) de votre application en utilisant des propriétés.



Méthodes et classes abstraites

- ▶ La notion de méthodes abstraites est utilisée lors de l'héritage afin de forcer les classes filles à implémenter une méthode qui ne le serait pas dans la classe parente.
 - ▶ Par extension, si une classe contient une méthode abstraite, la classe sera également abstraite car elle ne pourra pas être instanciée.
- ▶ La notion de classe abstraite existe en C++, Java, C#, ...
 - ▶ En java elle est surtout réalisée par la notion d'interface.
- ▶ Python ne supporte pas la notion de classe abstraite dans le cœur du langage, mais un module standard permet très facilement de la mettre en œuvre.



Méthodes et classes abstraites

- ▶ Il s'agit du module `abc` (*Abstract Base Classes*) qui fournit la classe `ABC` ainsi que le décorateur `abstractmethod`.
- ▶ La classe héritera de `ABC` et les méthodes abstraites seront définies avec le décorateur `abstractmethod` :

```
import abc

class A(abc.ABC):
    @abc.abstractmethod
    def do_smth(self):
        pass

class B(A):
    def do_smth(self):
        print("do_smth from B")
```

- ▶ Si les méthodes abstraites ne sont pas définies dans les classes filles, il ne sera pas possible d'instancier des objets de ces classes.
- ▶ Si aucune méthode n'est décorée, il n'y aura aucun effet.



Overriding et overloading

- ▶ Réécriture et surcharge de méthode avec Python.
- ▶ La réécriture (*overriding*) d'une méthode intervient lorsque l'on dérive une classe de base et que l'on désire écraser une méthode particulière de la classe parent.
 - ▶ Cela ne pose aucun problème et se réalise comme dans tout autre langage de programmation.
- ▶ La surcharge (*overloading*) permet de définir plusieurs méthodes avec le même nom mais agissant de manières différentes.
 - ▶ Cette notion est nécessaire pour les langages statiquement typés afin de permettre plus de... dynamisme.
 - ▶ Comme Python est dynamiquement typé et qu'il supporte les paramètres par défaut et les paramètres nommés pour les arguments des méthodes, il n'est pas nécessaire d'avoir un tel support.
 - ▶ Python **ne supporte pas** la surcharge de méthodes et, pour obtenir le même comportement, on utilise les paramètres par défaut et les paramètres nommés.



Réécriture des opérateurs

- ▶ Avec Python un opérateur tel que **+** correspond à un appel de méthode.
 - ▶ Tout étant objet, l'addition de 2 nombres entiers correspond à l'appel d'une méthode sur le premier opérande avec le second comme paramètre.
- ▶ Il est donc tout à fait possible de redéfinir ces opérations mathématiques dans une classe.
- ▶ Par exemple pour redéfinir l'opérateur **+**, il suffit de créer la méthode suivante :

```
def __add__(self, other)
```



Réécriture des opérateurs

► Par exemple :

```
class A:
    def __init__(self, x):
        self.x = x

    def __add__(self, other):
        return A(self.x + other.x)
```

- Ceci devrait couvrir la plupart des cas.
- Mais que ce passe-t-il dans le cas où les opérandes ne sont pas du même type ?
 - Par exemple si, dans le cas de la classe **A**, l'addition ne devait être gérée que en lui additionnant un nombre.

```
def __add__(self, n):
    return A(self.x + n)
```



Réécriture des opérateurs

- ▶ L'addition ne pourrait se faire que dans un seul sens :

```
a1 = A(12)
a2 = a1 + 33
a3 = 33 + a1
TypeError: unsupported operand type(s) for +: 'int' and 'A'
```

- ▶ C'est pourquoi Python offre également la méthode "inverse" `__radd__(self, other)` qui permet de gérer ce type de cas.
 - ▶ Celle-ci ne sera cependant appelée que si `__add__` a échoué.
- ▶ Et que se passe-t-il lorsque l'on utilise les opérateurs courts ?

```
a1 += 33
```

- ▶ Par défaut `__add__` ou `__radd__` est appelée, mais il existe en plus la méthode `__iadd__` qui, si elle est définie, sera appelée à la place.
- ▶ Ce qui peut parfois être intéressant.



Réécriture des opérateurs

- Liste non exhaustive des principaux opérateurs :

Opérateurs binaires	Méthodes
+	<code>__add__(self, other)</code>
-	<code>__sub__(self, other)</code>
*	<code>__mul__(self, other)</code>
/	<code>__truediv__(self, other)</code>
//	<code>__floordiv__(self, other)</code>
%	<code>__mod__(self, other)</code>
**	<code>__pow__(self, other)</code>

- Pour avoir une liste de tous les opérateurs :

<https://docs.python.org/3/library/operator.html>



Réécriture des opérateurs

Opérateurs unaires	Méthodes
<code>+</code>	<code>__pos__(self)</code>
<code>-</code>	<code>__neg__(self)</code>
<code>abs()</code>	<code>__abs__(self)</code>
<code>int()</code>	<code>__int__(self)</code>
<code>long()</code>	<code>__long__(self)</code>
<code>float()</code>	<code>__float__(self)</code>

Opérateurs de comparaisons	Méthodes
<code>==</code>	<code>__eq__(self, other)</code>
<code>!=</code>	<code>__ne__(self, other)</code>
<code><</code>	<code>__lt__(self, other)</code>
<code><=</code>	<code>__le__(self, other)</code>
<code>></code>	<code>__gt__(self, other)</code>
<code>>=</code>	<code>__ge__(self, other)</code>



Exercices 14

- Créez une classe **Currency** permettant de manipuler des sommes monétaires dans différentes monnaies et qui pourrait être utilisée de la manière suivante :

```
from currencies import Currency as C
v1 = C(20.50)
v2 = C(20.50, "EUR")
v3 = C(20.50, "USD")
print(v1 + v2)
43.28 CHF
print(v2 + v1)
38.95 EUR
print(v3 + 5)
25.50 USD
print(5 + v3)
25.50 USD
```



Manipuler les attributs

- ▶ Il est très facile d'accéder à un attribut dont le nom est connu :

```
print(object.foo)
object.foo = 33
```

- ▶ Mais est-il possible d'accéder à un attribut dont le nom serait variable ?
 - ▶ Oui, grâce à la fonction `getattr(obj, attr)`.
- ▶ Dans un tel cas il est généralement utile de pouvoir déterminer à l'avance si l'objet comporte l'attribut en question.
 - ▶ Ce qui se fait avec `hasattr(obj, attr)` qui retourne un booléen.
- ▶ Il sera alors également possible de :
 - ▶ Définir la valeur de cet attribut avec `setattr(obj, attr, value)`.
 - ▶ Effacer l'attribut avec `delattr(obj, attr)`.



Manipuler les attributs

```
a = A(12)
attr = "x"

if hasattr(a, attr):
    res = getattr(a, attr)
    setattr(a, attr, res * 2)
    delattr(a, attr)
```



Exercices 15

- ▶ Créez une interface à votre programme de l'exercice 8 afin de permettre à l'utilisateur d'avoir une liste de **Pet** et de demander des informations à propos d'une caractéristique que le **Pet** pourrait avoir.
- ▶ Par exemple pour savoir si le **Pet** a un âge est savoir quel est cet âge, l'utilisateur écrirait simplement **age** sur la ligne de commande de l'interface du programme.



Profiling et debugging



Introduction

- ▶ Le profilage du code consiste à obtenir des informations sur l'exécution du code :
 - ▶ Quelles sont les méthodes appelées.
 - ▶ Combien de fois une méthode particulière a-t-elle été appelée.
 - ▶ Combien de temps une méthode met à s'exécuter.
 - ▶ Quelle est la consommation mémoire du programme.
 - ▶ Quelle est la consommation CPU du programme.
- ▶ Généralement l'utilisation d'un profiler est liée à la volonté d'optimiser le code.
- ▶ L'idée est donc de déterminer ce qui ralentit le programme.
- ▶ Une bonne première approche est de mesurer le temps d'exécution de certaines parties du programme.



Fonctions temporelles

- ▶ Le module **time** de Python offre plusieurs fonctions permettant de mesurer le temps, telles que :
 - ▶ **monotonic()** **monotonic_ns()**
 - ▶ **perf_counter()** **perf_counter_ns()**
 - ▶ **process_time()** **process_time_ns()**
 - ▶ **time()** **time_ns()**
- ▶ Les versions ***_ns()** retournent un résultat entier en nanosecondes plutôt qu'un résultat de type **float**.
 - ▶ Permet d'éviter les erreurs d'arrondi du format **float**.
- ▶ Chaque horloge possède ses propres caractéristiques.



Fonctions temporelles

- ▶ La plus intéressante est **perf_counter()** car :
 - ▶ Elle est monotonique (ne peut pas aller en arrière).
 - ▶ Elle compte également le temps passé en mode *sleep*.
- ▶ **perf_counter()** ne donne pas l'heure, mais un temps défini par rapport à une référence aléatoire valable sur l'entier du système.
 - ▶ Seule la différence entre deux appels est donc significative.
 - ▶ Elle ne peut être utilisée que pour mesurer le temps qui s'écoule.



Mesure simple

- L'approche la plus simple pour mesurer le temps d'exécution d'un bout de code est de faire un appel avant, puis après, et mesurer la différence :

```
import time

START = time.perf_counter()

time.sleep(1)

STOP = time.perf_counter()

print(f"Execution time: {STOP-START:.6f}")
Execution time: 1.002001
```



Mesure efficace

- ▶ Pour aller plus loin une bonne approche consiste à définir une classe pour gérer le *timer* :
 - ▶ Celle-ci pourrait conserver l'utilisation de plusieurs *timers*, définir le texte à afficher, etc.
- ▶ Mais ce qui serait surtout intéressant serait la possibilité de définir à quel endroit les résultats seront fournis :
 - ▶ Affichage sur la sortie standard.
 - ▶ Sauvegarde dans un fichier.
 - ▶ Utilisation d'une fonction de *logging* précédemment définie.
- ▶ Pour faire simple on aurait le code de la slide suivante.



Mesure efficace

```
import time

class Timer:
    def __init__(self, logger=print):
        self._start = None
        self._logger = logger

    def start(self):
        if self._start:
            raise Error("Timer is running")
        self._start = time.perf_counter()

    def stop(self):
        if not self._start:
            raise Error("Timer is stopped")
        elapsed = time.perf_counter() - self._start
        self._start = None
        self._logger(f"Elapsed time: {elapsed}")
        return elapsed
```



Mesure efficace

- ▶ Que l'on peut utiliser directement :

```
import time
from timer import Timer

t = Timer()
t.start()
time.sleep(1)
t.stop()
```

- ▶ Ou en l'initialisant avec une fonction de *logging* dans un fichier :

```
def log_in_file(msg):
    with open("/tmp/mylog", 'wt') as fd:
        print(msg, file=fd)

t = Timer(log_in_file)
```



Mesure efficace

- Ou avec une logique de *logging* personnelle :

```
import time
import logging
from timer import Timer

logging.basicConfig(format="%(asctime)s [%(filename)s]
%(levelname)s: %(message)s (Line: %(lineno)s)")

t = Timer(logging.warning)
```



En utilisant un *context manager*

- ▶ Avec une classe permettant de gérer la logique du *timer* il est possible de l'utiliser de manière plus efficace en définissant, par exemple, un *context manager*.
- ▶ Suivant la logique du *duck typing*, il suffit d'ajouter deux méthodes :

```
def __enter__(self):  
    self.start()  
    return self  
  
def __exit__(self, *exec_info):  
    self.stop()
```

- ▶ Et l'on peut maintenant faire simplement :

```
with Timer() as t:  
    time.sleep(1)
```



En utilisant un décorateur

- ▶ L'intérêt du *context manager* est qu'il permet de tracer le temps d'exécution de plusieurs instructions très simplement.
- ▶ Mais pour tracer le temps d'exécution des appels de fonctions (ou de méthodes) l'approche n'est pas la plus propre.
 - ▶ Ni d'ailleurs la plus simple à intégrer.
- ▶ Une meilleure approche consiste à définir un décorateur.
 - ▶ Celui-ci permettra de calculer très simplement le temps mis pour chaque fonction du programme à s'exécuter.

```
@timer
def do_something():
    time.sleep(1)

do_something()
```

- ▶ Et pour déterminer le temps d'exécution du programme complet, il suffit d'intégrer celui-ci dans une fonction.



En utilisant un décorateur

- Code du *timer* en tant que décorateur :

```
import functools
import time

def timer(func):

    @functools.wraps(func)
    def inner_timer(*args, **kwargs):
        start = time.perf_counter()
        ret = func(*args, **kwargs)
        stop = time.perf_counter()
        elapsed = stop - start
        print(f"Elapsed time: {elapsed:.6f} seconds")
        return ret

    return inner_timer
```



Mesures macro ou mesures micro

- ▶ Les approches précédentes permettent de se faire une idée du temps d'exécution du code de manière macroscopique.
- ▶ Après avoir déterminé les fonctions ou les instructions qui posent problème, il s'agit de proposer de nouvelles approches et de les comparer afin de déterminer celles qui sont les plus efficaces.
- ▶ Il est possible de le faire avec les *timers* développés précédemment, mais Python fournit un outil dédié à ce type de tests.
 - ▶ Il s'agit du module `timeit`.
- ▶ Celui-ci permet d'effectuer plusieurs fois un bout de code ou une fonction et fournit la moyenne du temps d'exécution.
- ▶ Son utilisation est un peu spéciale :
 - ▶ Le code à exécuter et les préconditions (*setup*) sont fournis sous forme de **chaînes de caractères**.



Comparer les temps d'exécution

- ▶ Une manière simple de l'utiliser est en ligne de commande.
- ▶ Par exemple pour déterminer si la recherche d'un caractère dans une chaîne est plus rapide avec l'instruction `in` ou avec la méthode `find()` :

```
[pny]$ python -m timeit \  
> --setup 't="Ils ont des chapeaux ronds";c="x"' "c in t"  
10000000 loops, best of 5: 20.1 nsec per loop  
  
[pny]$ python -m timeit \  
> --setup 't="Ils ont des chapeaux ronds";c="x"' "t.find(c)"  
5000000 loops, best of 5: 61.9 nsec per loop
```

- ▶ Il semble que l'instruction `in` soit 3 fois plus rapide que la méthode `find()` ...



Comparer les temps d'exécution

- ▶ **timeit** fournit la fonction **timeit** permettant d'effectuer des mesures depuis l'interpréteur Python ou depuis un script :

```
from timeit import timeit

e = timeit("c in t", 't="Ils ont des chapeaux ronds";c="x"')
print(e / 1000000)

e = timeit("t.find(c)", 't="Ils ont des chapeaux ronds";c="x"')
print(e / 1000000)
```

- ▶ La division permet d'avoir le temps d'exécution pour une exécution car **timeit** exécute le code **1000000** fois (par défaut).
- ▶ **timeit** fournit également la fonction **repeat** qui ne fait qu'appeler **timeit** un certain nombre de fois (par défaut **5x**) et fournit une liste des résultats.



Comparer les temps d'exécution

- ▶ `timeit` et `repeat` s'exécutent de manière spéciale.
- ▶ Ils se trouvent dans leur propre espace mémoire.
- ▶ De ce fait ils n'ont pas connaissance des fonctions et autres variables définies au même niveau que leur exécution.
- ▶ Le code suivant, par exemple, ne fonctionne pas :

```
from timeit import timeit

def recu_fact(val):
    if not val:
        return 1
    return val * recu_fact(val - 1)

timeit("recu_fact(100)")
```

- ▶ **`NameError: name 'recu_fact' is not defined.`**



Comparer les temps d'exécution

- ▶ Il est nécessaire d'utiliser le paramètre de setup afin de charger les éléments nécessaires.
- ▶ Ceux-ci seront chargés depuis le module `__main__` de la manière suivante :

```
timeit("recu_fact(100)", "from __main__ import recu_fact")
```

- ▶ Dans le cas où des variables seraient également nécessaires, elles seraient chargées de manière identique :

```
timeit("recu_fact(nb)", "from __main__ import recu_fact, nb")
```

- ▶ `timeit` a permis, par exemple, de déterminer quelle était la manière la plus efficace de concaténer deux dictionnaires.



Comparer les temps d'exécution

- ▶ A noter que `timeit` est très simplement accessible depuis le shell **IPython** (notebook **Jupyter** et IDE Spyder).
- ▶ Il suffit d'ajouter la commande magique `%timeit` en début de ligne :

```
In [1]: d1 = {f"d1_{i}":i for i in range(100)}  
In [2]: d2 = {f"d2_{i}":i for i in range(100)}  
In [3]: def merge():  
...:     for k, v in d2.items():  
...:         d1[k] = v  
...:  
In [4]: %timeit merge()  
4.08 µs ± 14.5 ns per loop (mean ± std. dev. ... each)
```

- ▶ Dans un notebook Jupyter ou dans l'interpréteur de Spyder la commande `%%timeit` permet de mesurer le temps d'exécution de toute la cellule.



Utiliser un profiler

- ▶ **timeit** ainsi que les *timers* sont intéressants pour avoir une idée du temps d'exécution de certaines parties du code.
- ▶ Mais lorsque l'application complète semble lente il serait intéressant de pouvoir en déterminer les *bottlenecks*.
- ▶ Dans ce cas l'utilisation d'un profiler est plus indiquée.
- ▶ Python fournit un tel outil avec le module **cProfile** dont l'utilisation la plus simple consiste à exécuter le programme en ligne de commande en chargeant le module :

```
[pny]$ python -m cProfile --sort calls script.py
```

- ▶ La sortie fournit des informations sur le temps d'exécution et le nombre d'appels de chaque fonction.



Utiliser un profiler

- ▶ La meilleure approche consiste à sauvegarder le résultat dans un fichier à l'aide de l'option **-o**

```
[pny]$ python -m cProfile -o factoriel.prof factoriel.py
```

- ▶ Le fichier d'output (**factoriel.prof**) est de type binaire et on y accède à l'aide du module **pstats** :

```
[pny]$ python -m pstats factoriel.prof
factoriel.prof% help

Documented commands (type help <topic>):
=====
EOF  add  callees  callers  [...]  reverse  sort  stats  strip
```

- ▶ Celui-ci fournit un certain nombre de fonctionnalités pour trier les résultats, afficher l'ordre des appels, etc.



Utiliser un profiler

- Dans le cas de l'exemple suivant (calcul factoriel de manière itérative et récursive) :

```
factoriel.prof% sort ncalls
factoriel.prof% stats 3
Wed Nov 10 14:20:25 2021 factoriel.prof
  55153 function calls (203 primitive calls) in 0.018 seconds
  Ordered by: call count
  ncalls tottime percall cumtime percall filename:lineno(function)
55050/100   0.013   0.000   0.013   0.000 factoriel.py:12(recu_fact)
    100   0.005   0.000   0.005   0.000 factoriel.py:3(iter_fact)
     1    0.000   0.000   0.018   0.018 {built-in ... builtins.exec}
```

- On constate que la majorité du temps ainsi que près de **100%** des appels sont dus à la fonction **recu_fact**.
- Ce qui permet de se faire une idée de ce qui pourrait causer des lenteurs dans l'exécution du programme.



Utiliser un profiler

- ▶ Le profiler peut être utilisé pour détecter le code qui pourrait constituer un goulot d'étranglement et qui pourrait constituer une cible d'optimisation.
- ▶ Mais également pour simplement mieux comprendre le fonctionnement du programme.
 - ▶ Par exemple avec les commandes **calleeps** et **callers** qui montrent quelle fonction est appelée par quelle autre fonction ou appelle quelle autre fonction du programme.
- ▶ Le profiler permet par exemple de vérifier l'*overhead* dû à l'utilisation d'un *timer* :
 - ▶ Pour cela on utilise la capacité de filtrage de la commande **stats** en précisant le nom du module (**timer.py**) dans lequel est défini le *timer* :

```
yop.prof% stats timer.py
```



Debugging: Tracing

- ▶ La première étape consiste à tracer l'exécution du code à l'aide de `print()`.
 - ▶ Ce qui est souvent suffisant pour de petits scripts.
- ▶ Il existe cependant beaucoup d'autres outils qui permettent de faciliter ces manipulations même pour de petits scripts.
- ▶ Par exemple **IceCream** (<https://github.com/gruns/icecream>) qui permet de simplifier les `print()` :

```
counter = 3
print("Value of counter:", counter)
Value of counter: 3

from icecream import ic
ic(counter)
ic| counter: 3
```



Debugging: Tracing

- ▶ Une autre approche très utile et qui devrait d'ailleurs toujours être intégrée dans le code est l'utilisation du logging, que l'on a vu précédemment.
- ▶ Et lorsque le traçage du code n'est pas suffisant ou que le programme grandit et qu'il devient compliqué de mettre des `print()` partout, on peut utiliser :
 - ▶ **PySnooper** qui est une sorte de "`set -x`" pour Python.
 - Il offre un décorateur qui permettra de tracer les appels d'une fonction/méthode.
 - ▶ **snoop** est également intéressant.
 - Basé sur **PySnooper** mais avec plus de fonctionnalités (sortie plus verbeuse, colorisation syntaxique et intégration de sa propre version de **IceCream**).



Debugging: Tracing

► **PySnooper** :

```
from pysnooper import snoop  
  
@snoop()  
def do_smth(...):
```

► **snoop** :

```
import snoop  
  
@snoop  
def do_smth(...):
```



Debugging: Debugging

- ▶ Généralement c'est le dernier recours, lorsque le traçage du programme n'a pas permis de déterminer le problème.
- ▶ La base du *debugging* est fournie par le module **pdb**.
- ▶ Il s'agit du debugger officiel de Python.
- ▶ Il permet de debugger le code en ligne de définir des *breakpoints*, de suivre les appels de fonctions, etc.
- ▶ Mais ce n'est pas le plus simple à utiliser.
- ▶ Les IDEs (**PyCharm**, **Spyder**) fournissent un débogueur avec une interface graphique qui les rendent très facile à prendre en main.
- ▶ Sinon un débogueur intéressant est **PuDB**.
 - ▶ Celui-ci fournit une interface graphique dans une console, ce qui le rend utilisable très simplement et même à distance.



Testing



Qu'est-ce que le testing ?

- ▶ Le but des phases de tests est d'évaluer un élément d'un système ou un système complet afin de vérifier qu'il correspond aux attentes ou aux exigences.
- ▶ Ces vérifications sont effectuées à l'aide de tests.
- ▶ On distingue différents niveaux de tests, typiquement :
 - ▶ Les tests unitaires.
 - Le but est de tester un élément particulier (classe, fonction) d'un système.
 - ▶ Les tests d'intégration.
 - Le but est de tester le fonctionnement de plusieurs éléments ensemble.
 - ▶ Les tests système.
 - Le but est de tester le fonctionnement du système dans son ensemble.
 - ▶ Les tests d'acceptance.
 - Le but est de tester le système en regard des exigences des clients.



Qu'est-ce que le testing ?

- ▶ Les tests d'intégration et les tests unitaires sont généralement automatisables.
 - ▶ Ce qui n'est pas le cas des tests systèmes et d'acceptance.
- ▶ Raison pour laquelle les *frameworks* de test permettent généralement d'effectuer des tests unitaires et des tests d'intégration.
- ▶ Les tests d'intégration sont sensibles aux modifications de code, mais ne peuvent généralement pas être effectués aussi rapidement que les tests unitaires.
 - ▶ Il faut également en utiliser abondamment.
 - ▶ Mais en premier lieu, il faut s'assurer que les tests unitaires passent avec succès.



Pytest

- ▶ Il existe différents *frameworks* de test :
 - ▶ **Unittest**, **Pytest**, **Tox**, **Nose**, ...
- ▶ **Unittest** est intégré à Python, il constitue un bon candidat.
- ▶ Cependant sa syntaxe n'est pas très *pythonique* et la définition de tests nécessite d'hériter de classes spécifiques.
 - ▶ Il n'est pas possible de créer des fonctions de test.
- ▶ Le *framework* le plus utilisé est **Pytest**.
- ▶ Une des fonctionnalités intéressantes de **Pytest** est qu'il est capable de découvrir par lui-même les fichiers contenant des tests ainsi que les tests à exécuter.



Pytest

- ▶ Pour cela il suffit que le nom du fichier commence par **test_**.
- ▶ Ensuite, à l'intérieur de ce fichier, Pytest exécutera :
 - ▶ Les fonctions dont le nom commence par **test**.
 - ▶ Les méthodes dont le nom commence par **test** et qui se trouvent dans une classe dont le nom commence par **Test**.
- ▶ Le souligné n'est pas nécessaire, mais pour les fonctions et les méthodes, l'ajouter rend le tout plus *pythonique* et plus lisible.



Déroulement d'un test

- ▶ Le déroulement d'un test est généralement le suivant :
 - ▶ Préparation des données à fournir au système.
 - ▶ Identification des résultats attendus.
 - ▶ Récupération des résultats fournis par le système.
 - ▶ Comparaison des résultats fournis avec les résultats attendus.
- ▶ Le dernier point est centré autour du mot clé **assert**.
 - ▶ Celui-ci permet de tester si une condition retourne **True**.
- ▶ Si la condition retourne **False** alors l'exception **AssertionError** est lancée.



Premier test

- ▶ Ainsi si le fichier `functions.py` contient :

```
def double_it(n):  
    return n * 2
```

- ▶ Le fichier `test_functions.py` pourrait contenir :

```
from functions import double_it  
  
def test_double_it():  
    result = double_it(2)  
    assert result == 4
```



Premier test

- ▶ Et l'exécution des tests depuis le terminal avec l'une des commandes suivantes :

```
[pny]$ python -m pytest .  
[pny]$ pytest .
```

- ▶ En préférant l'approche utilisant des fonctions.
 - ▶ Plus simple.
- ▶ L'option **-v** permettra d'avoir une sortie plus verbeuse.



Premier test

- ▶ Dans le cas d'une exécution avec succès on aura simplement :

```
===== test session starts =====  
platform linux -- Python 3.11.2, pytest-8.2.0, pluggy-1.5.0  
rootdir: /tmp/test  
collected 1 item  
  
test_functions.py . [100%]  
===== 1 passed in 0.01s =====
```

- ▶ Le point (.) qui suit le nom du fichier montre combien de tests on été exécutés.
 - ▶ L'option **-v** permet d'avoir une sortie plus verbeuse.



Premier test

- Dans le cas d'un échec la sortie est plus verbeuse :

```
===== test session starts =====
platform linux -- Python 3.11.2, pytest-8.2.0, pluggy-1.5.0
rootdir: /tmp/test
collected 1 item

test_functions.py F [100%]
===== FAILURES =====
_____ test_double_it _____
      def test_double_it():
>         assert double_it(2) == 5
E         assert 4 == 5
E         + where 4 = double_it(2)
test_functions.py:5: AssertionError
===== short test summary info =====
FAILED test_functions.py::test_double_it - assert 4 == 5
===== 1 failed in 0.02s =====
```



Tester les exceptions

- ▶ Si une exception se produit lors d'un test, celui-ci aura alors automatiquement échoué.
 - ▶ Ce qui est logique.
- ▶ Cependant il est parfois utile de pouvoir tester que l'exécution d'un test produit effectivement une exception.
- ▶ Par exemple on pourrait vouloir tester qu'une division par 0 produit bien une exception de type **ZeroDivisionError**.
- ▶ Par exemple la fonction suivante :

```
def divide(a, b):  
    return a / b
```



Tester les exceptions

- ▶ Que l'on pourrait vouloir tester avec :

```
def test_divide():  
    result = divide(10, 5)  
    assert result == 2  
  
def test_divide_by_zero():  
    result = divide(10, 0)  
    assert True
```

- ▶ Mais bien que l'on retourne **True**, le test échoue.
- ▶ L'approche consiste à tester que l'exception générée est bien celle attendue.



Tester les exceptions

- Ce qui se fait avec un *context manager* de la manière suivante :

```
import pytest

def test_divide_by_zero():
    with pytest.raises(ZeroDivisionError):
        divide(10, 0)
```



Afficher les sorties

- ▶ Par défaut les sorties produites, typiquement l'affichage de texte avec `print`, ne seront pas visibles dans le résultat de l'exécution des tests.
 - ▶ Que ces sorties soient générées par la fonction à tester ou par la fonction de test.
- ▶ Pour que Pytest les affiche on utilise l'option `-s` :

```
[pny]$ pytest -s .
```



Organisation de l'arborescence

- ▶ Pour faciliter la compréhension du code on essaie d'avoir une bonne organisation des fichiers dans un projet.
- ▶ Dans ce sens on sépare le code des tests, par exemple avec une organisation telle que la suivante :

```
.  
├── source  
│   ├── __init__.py  
│   └── functions.py  
└── tests  
    ├── __init__.py  
    └── test_functions.py
```

- ▶ Dans les fichiers de tests, le chargement des modules du code est effectué à l'aide d'un chemin absolu :
 - ▶ Car le répertoire du projet n'est pas un package

```
from source.functions import divide
```



Annexes

Expressions régulières

Testing



Expressions régulières

Caractères, ensembles, quantificateurs

Alternatives, groupes, assertions



Expressions régulières

- ▶ Une expression régulière représente un motif (*pattern*) utilisant ou pas des caractères spéciaux permettant de rechercher des correspondances de texte de manière générique.
 - ▶ En anglais on parle de ***pattern matching***.
- ▶ Une expression régulière est construite en définissant :
 - ▶ Des caractères, normaux ou spéciaux,
 - ▶ Des ensembles de caractères,
 - ▶ Des groupes,
 - ▶ Des alternatives,
 - ▶ Auxquels on adjoints ou pas un multiplicateur
 - ▶ Ainsi que des assertions, permettant de marquer une position dans la chaîne, si cela est nécessaire.



Expressions régulières : Les caractères

- ▶ Dans le cas général un caractère vaut pour lui même.
 - ▶ C'est-à-dire qu'il sera recherché tel quel dans la chaîne.
 - ▶ Le motif **"python"** correspond à la suite de caractères **"python"** n'importe où dans la chaîne de recherche.
- ▶ Certains caractères sont spéciaux et ont une signification particulière dans le motif.
 - ▶ Si l'on désire rechercher un tel caractère il est nécessaire de modifier sa signification en ajoutant un *backslash* (\) devant.
- ▶ Ces caractères spéciaux sont les suivants :

\ . [] ^ \$ * + ? { } | ()

- ▶ Le point (.) représente **n'importe quel caractère**.
- ▶ Le motif **' . '** retournera toutes les chaînes de caractères qui contiennent au moins **1** caractère, quel que soit ce caractère.



Expressions régulières : Les ensembles

- ▶ Les crochets (**[** et **]**) permettent de créer des ensembles de caractères.
- ▶ Un ensemble ne représente qu'**un seul** caractère qui peut être n'importe quel caractère de l'ensemble.
 - ▶ Le tiret (**-**) permet de définir un range de caractères.
 - ▶ Le circonflexe (**^**) permet d'inverser la sélection.

Ensemble	Signification
[abc]	Un seul caractère qui peut être a ou b ou c .
[a-z]	Une lettre minuscule.
[a-z-]	Une lettre minuscule ou le tiret.
[am-pA-Z]	Les caractères a , m à p et A à Z .
[^ab]	N'importe quel caractère sauf a ou b .



Expressions régulières : Les ensembles

- Pour simplifier la définition des ensembles classiques il existe des notations spéciales :

Ensemble	Signification
<code>\d</code>	N'importe quel caractère représentant un chiffre. Correspond en ASCII à <code>[0-9]</code> .
<code>\s</code>	N'importe quel caractère d'espacement. Correspond en ASCII à <code>[\t\n\r\f\v]</code> .
<code>\w</code>	N'importe quel caractère constituant un mot. Correspond en ASCII à <code>[a-zA-Z0-9_]</code> .

- Pour définir les ensembles inverses il suffit d'utiliser des majuscules :
 - `\D` Un caractère non numérique.
 - `\S` Tout sauf un caractère d'espacement.
 - `\W` Un caractère non alphabétique et pas le souligné.



Expressions régulières : Les quantificateurs

- ▶ Ils s'appliquent au motif atomique qui les précède.
 - ▶ Donc à un caractère ou un ensemble défini précédemment.
- ▶ Ils permettent de définir combien de fois la définition précédente peut se répéter de suite.

Quantificateur	Signification
*	N'importe combien de fois. De 0 à l'infini.
+	Au moins une fois. De 1 à l'infini.
?	0 ou 1 fois
{m}	Exactement m fois.
{m,n}	Entre m et n fois. Si m est omis cela signifie au plus n fois ({0,n}). Si n est omis cela signifie au moins m fois ({m,infini}).



Expressions régulières : Les quantificateurs

- ▶ Les quantificateurs sont gourmands par défaut.
 - ▶ C'est-à-dire qu'ils veulent correspondre au maximum de caractères possible.
- ▶ Pour les rendre non gourmands on leur adjoint un "?":

Motif	Correspondance sur '<a> ccccc '
<.*>	<a> ccccc
<.*?>	<a>
<.+>	<a> ccccc
<.+?>	<a>
c{3,5}	cccccc
c{3,5}?	ccc



Expressions régulières : Les alternatives

- ▶ Avec les définitions vues jusque là il est difficile matcher deux mots différents.
- ▶ Un ensemble permet de définir un choix de caractères.
- ▶ Pour définir un choix de mots, ou plus génériquement, un choix entre plusieurs suites de caractères on utilise des alternatives.
- ▶ Que l'on définit à l'aide du caractère pipe (|) :

```
"Julien|Prunille|Mickey"
```

- ▶ Permet de rechercher la chaîne **"Julien"** ou la chaîne **"Prunille"** ou la chaîne **"Mickey"**.



Expressions régulières : Les groupes

- ▶ Les groupes permettent de délimiter une partie d'une expression régulière agissant d'une manière définie.
- ▶ Par exemple en permettant de mixer des alternatives dans une expression régulière avec d'autres caractères :

```
"(Pierre|Paul) Bolomey"
```

- ▶ Ou pour définir un multiplicateur sur plus d'un caractère.
 - ▶ Où le terme "caractère" est à prendre au sens des expressions régulières. Les alternatives permettant de regrouper des caractères et de les considérer comme un motif atomique.
 - ▶ On peut chercher une vache qui meule longtemps ou une vache qui meugle souvent :

```
"meuh{5,6}"           # La quelle est-ce ?  
"(meuh){5,6}"          # La quelle est-ce ?
```



Expressions régulières : Les groupes

- ▶ Les groupes permettent également les répétitions.
 - ▶ Le texte correspondant au groupe pourra être réutiliser plus loin dans l'expression régulière.
- ▶ Les groupes permettent également l'extraction de données.
 - ▶ Le texte correspondant au groupe pourra être récupéré dans la suite du programme.
- ▶ Exemple :

```
"<(\w+?)>(.+)</\1>"
```

- ▶ Permettra d'extraire dans un deuxième groupe le texte d'un élément HTML défini par son tag ouvrant et son tag fermant récupéré à l'aide de la définition du premier groupe.



Expressions régulières : Les assertions

- Les assertions permettent de positionner le motif par rapport à la chaîne de recherche :

Assertion	Signification
^	Marque le début de la chaîne.
\$	Marque la fin de la chaîne.

```
"^(Julien|Prunille).*over$"
```

- La chaîne doit commencer par **Julien** ou **Prunille**, puis contenir n'importe quel caractère, **0** ou plusieurs fois, puis se terminer par le mot **over**.
- Notez que l'utilisation de l'assertion de début de chaîne n'est pas nécessaire avec la méthode **match**.



Testing

Tester les classes

Fixtures

Initialisation et nettoyage

Paramétrage

Simulation



Test de classes

- ▶ Pour tester des classes il peut paraître logique de définir des tests sous forme de classes également.
- ▶ Soit une classe **Pet** :

```
class Pet:
    def __init__(self, name, weight):
        self.weight = weight
        self.name = name.upper()

    def eat(self, quantity):
        self.weight += 0.1 * quantity
        return self.weight

    def get_name(self):
        return self.name
```



Test de classes

- Le fichier permettant de tester le code pourrait être :

```
class TestPet:
    def test_eat(self):
        toby = Pet("Tobby", 2)
        result = toby.eat(2)
        assert result == 2.2

    def test_name(self):
        toby = Pet("Tobby", 2)
        result = toby.get_name()
        assert result == "TOBBY"
```

- Pour chaque test il faut créer **tobby**.
 - Ces conditions d'initialisation nécessaires sont redondantes.
 - Et donc automatiquement source d'erreurs.



Test de classes

- ▶ Avec l'approche basée classes, Pytest exécute automatiquement certaines méthodes avant et après l'exécution d'un test.
- ▶ Ce qui permet d'effectuer les initialisations nécessaires ainsi que les nettoyages après un test afin que chaque test soit effectué dans des conditions optimales.
- ▶ Ces deux méthodes (initialisation et nettoyage) se définissent de cette manière :
 - ▶ Avec un affichage pour voir leur exécution.

```
def setup_method(self, method):  
    print("SETUP")  
    self.tobby = Pet("Tobby", 2)  
  
def teardown_method(self, method):  
    print("CLEANING")  
    self.tobby = None
```



Test de classes

- Ce qui simplifie les tests et donne finalement le test :

```
class TestPet:
    def setup_method(self, method):
        print("SETUP")
        self.tobby = Pet("Tobby", 2)

    def teardown_method(self, method):
        print("SETUP")
        self.tobby = None

    def test_eat(self):
        result = self.tobby.eat(2)
        assert result == 2.2

    def test_name(self):
        result = self.tobby.get_name()
        assert result == "TOBBY"
```



Fixtures

- ▶ Pytest fournit une approche plus simple pour définir des pré-conditions d'un test.
- ▶ Il s'agit des **fixtures**.
- ▶ L'intérêt étant que celles-ci peuvent s'utiliser avec des fonctions de test.
- ▶ Une fixture représente une fonction qui pourra être injectée dans une fonction de test.
- ▶ Pour définir une fixture on utilise un décorateur :

```
@pytest.fixture
```



Fixtures

- Les tests précédents peuvent être réécrit de cette manière :

```
import pytest

@pytest.fixture
def toby():
    return Pet("Tobby", 2)

def test_eat(toby):
    result = toby.eat(2)
    assert result == 2.2

def test_name(toby):
    result = toby.get_name()
    assert result == "TOBBY"
```

- C'est bien plus beau et bien plus simple.



Appel de fixtures

- ▶ Les fixtures peuvent s'utiliser entre-elles.
- ▶ L'approche est cependant spécial :
 - ▶ Elles doivent être passées en paramètre à une autre fixture.
 - ▶ Il n'est pas possible d'appeler une fixture depuis une fixture.
- ▶ Par exemple pour créer une fixture qui renverrait 2 **Pet** :

```
@pytest.fixture
def toby_and_felix(tobby):
    return toby, Pet("Felix", 4)
```

- ▶ Qui pourrait être utilisée de la manière suivante :

```
def test_is_not_equal(tobby_and_felix):
    toby, felix = toby_and_felix
    assert toby != felix
```



Simulation des phases de setup et teardown

- ▶ L'approche objet offre la possibilité de nettoyer l'environnement après l'exécution d'un test.
 - ▶ Ce qui ne semble pas être le cas avec les fixtures.
- ▶ En fait si, en utilisant le mot clé **yield**.
- ▶ Lors de l'exécution d'une *yield fixture*, Pytest se comporte de la manière suivante :
 - ▶ Il exécute les instructions jusqu'au mot clé **yield**.
 - ▶ Il exécute le test.
 - ▶ Il retourne dans la fixture afin d'exécuter les instructions qui suivent le mot clé **yield**.



Simulation des phases de setup et teardown

- ▶ Ainsi une fixture avec nettoyage sera du style :

```
@pytest.fixture
def setup_teardown():
    print("Setup")
    yield
    print("Teardown")
```

- ▶ Et lorsqu'elle est exécutée simplement avec :

```
def test_setup_teardown(setup_teardown):
    print("Do test")
```

- ▶ Le résultat sera :

```
tests/test_pet.py Setup
Do test
.Teardown
```



Simulation des phases de setup et teardown

- ▶ Le mot clé **yield** peut également être intéressant pour définir un contexte d'exécution pour un test.
- ▶ Par exemple si un test nécessite une connexion sur une DB, il suffit de créer la connexion avec un *context manager* et de la retourner avec **yield** :

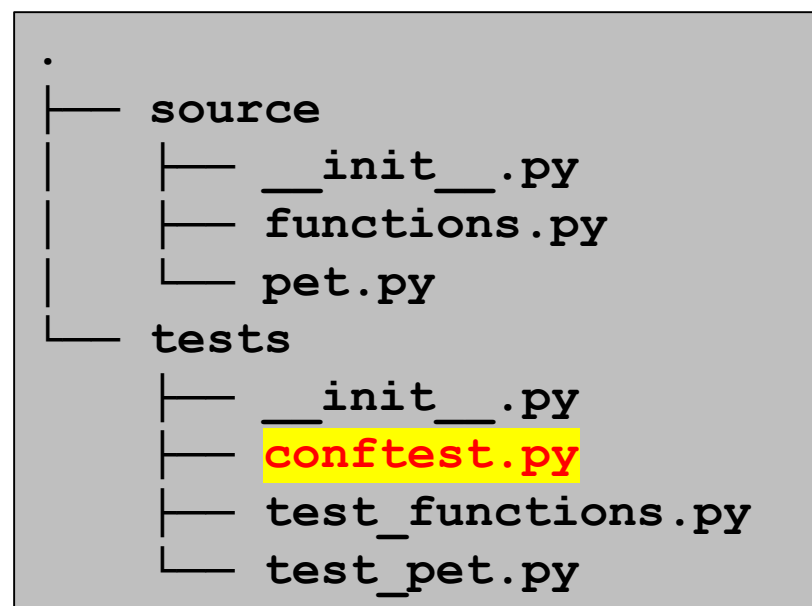
```
@pytest.fixture
def setup_db_connection():
    with sqlite3.connect(db) as conn:
        yield conn
```

- ▶ La sortie du *context manager* sera effectuée après le test.
 - ▶ Ce qui terminera la connexion à la DB.



Partager les fixtures

- ▶ Les fixtures définies dans un fichier de test ne seront accessibles que dans ce fichier.
- ▶ Pour pouvoir les partager avec d'autres fichiers, il suffit de les déplacer dans un fichier portant le nom **conftest.py**.
- ▶ Elles seront automatiquement accessibles depuis tous les fichiers qui se trouvent dans le même répertoire ou dans les sous-répertoires :
 - ▶ Et cela sans nécessiter d'import particulier.





Paramétrer les tests

- ▶ Généralement les tests sont effectués avec différentes valeurs.
 - ▶ Celles-ci proviennent par exemple d'une analyse des classes d'équivalence et des valeurs limites.
- ▶ Définir un test par set de valeur semble quelque peu *overkill*.
- ▶ C'est pourquoi Pytest permet de définir des tests paramétrés à l'aide du décorateur :

```
@pytest.mark.parametrize
```

- ▶ Il permet d'associer des données avec les paramètres d'un test.
- ▶ Ces données sont fournies sous la forme d'une **liste de tuples**.



Paramétrer les tests

- ▶ Par exemple pour tester la fonction suivante :

```
def add_two(a, b):  
    return a + b
```

- ▶ Le test sera défini de cette manière :

```
data = [(1, 2, 3), (-1, 1, 0), (0, 0, 0)]  
  
@pytest.mark.parametrize("a, b, expected", data)  
def test_add_two(a, b, expected):  
    assert add_two(a, b) == expected
```

- ▶ Pytest effectuera le test pour chacun des 3 sets de données.



Mocking (simulation)

- ▶ Lorsqu'un test utilise un service externe il peut être intéressant de simuler ce service.
- ▶ Généralement pour des questions de stabilité des données retournées ou à cause du délai de réponse.
 - ▶ Par exemple lors de la récupération de données depuis une DB ou depuis une API web.
- ▶ La mise en place d'un tel support (simulateur) est utile dans le cas de tests d'intégration.
 - ▶ Ce qui est testé n'est pas la connexion avec le service, mais celle-ci est nécessaire pour le fonctionnement global du test.



Mocking : Simuler l'accès à une DB

- ▶ Soit la fonction `get_watch(id)` qui retourne les données d'une montre depuis une DB.
 - ▶ Elle se trouve dans un fichier `source/service.py`.
- ▶ `get_watch()` est la fonction à simuler afin de s'abstraire de la connexion à la DB.
- ▶ Pour créer le simulateur on utilise la librairie `mock` du package `unittest` de Python.
- ▶ Celui-ci fournit un décorateur permettant de définir la fonction à simuler.



Mocking : Simuler l'accès à une DB

- ▶ La fonction de test (`tests/test_service.py`) est :

```
from source.service import get_watch
from unittest import mock

@mock.patch("source.service.get_watch")
def test_get_watch(mock_get_watch):
    mock_get_watch.return_value = {'id':1, 'name':"Pluto"}
    watch = get_watch(1)
    assert watch['name'] == "Pluto"
```

- ▶ Le décorateur définit la fonction à simuler.
- ▶ Il crée un simulateur qui est passé à la fonction de test.
- ▶ Une valeur de retour est définie pour le simulateur.
- ▶ Lorsque la fonction à simuler est appelée, c'est le simulateur qui retourne la valeur qui lui a été précédemment enregistrée.



Mocking : Simuler l'accès à une API web

- ▶ Dans le cas où `get_watch` effectue une requête sur une API pour récupérer les informations de la montre, il est possible de simuler uniquement la requête.
- ▶ Qui serait typiquement exécutée par `requests.get()`.

```
import requests

def get_watch(id):
    res = requests.get("http://shop.[...]/1")
    watch = res.json()
    return watch
```

- ▶ Le simulateur sera transmis à la fonction `get_watch` lors de l'appel de celle-ci.



Mocking : Simuler l'accès à une API web

- ▶ La création du simulateur est un petit peu plus complexe car il doit retourner un objet :
- ▶ Mais `unittest.mock` fournit tout le nécessaire.

```
@mock.patch("requests.get")
def test_get_watch(mock_get_watch):
    mock_response = mock.Mock()
    mock_response.status_code = 200
    mock_resonse.json.return_value = {
        'id': 1,
        'name': "Pluto"
    }
    mock_get_watch.return_value = mock_resonse
    watch = get_watch(1)
    assert watch['name'] == "Pluto"
```